

Java Backend Interview Preparation Guide

100 Conceptual Questions + Development → Deployment Reference

Core Java · Spring Boot · JPA · REST · Microservices · Kafka · Redis · AWS

Part 1. Core Java & OOP

Q1. Explain OOP pillars with a real e-commerce example.

SIMPLE ANSWER

Encapsulation: keep object data protected and allow controlled access through methods—for example, Order should not allow invalid status changes directly; `updateStatus()` validates the allowed transition. Inheritance: `BasePaymentGateway` extended by `StripeGateway` and `RazorpayGateway`. Polymorphism: `PaymentService` calls `process()` on any gateway implementation. Abstraction: `OrderRepository` interface hides JPA details. Together they keep checkout logic maintainable as payment providers change.

STEP-BY-STEP

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

REAL PROJECT EXAMPLE

`PaymentService.process(order, gateway)` works with any `PaymentGateway` implementation without knowing Stripe vs Razorpay internals.

INTERVIEW KEYWORDS

Encapsulation | Inheritance | Polymorphism | Abstraction

Interview Tip: For encapsulation, mention validation in methods—not just getters/setters.

One-Line Interview Answer: OOP uses encapsulation, inheritance, polymorphism, and abstraction to structure maintainable backend code.

Q2. Difference between abstract class and interface in Java.

SIMPLE ANSWER

An abstract class can have state, constructors, and both abstract and concrete methods—use when subclasses share common behavior (e.g., `BaseEntity` with `id`, `createdAt`). An interface defines a behavioral contract—use for capabilities like `Payable` or `Notifiable`. Since Java 8, interfaces may include default, static, and private methods, but they do not hold per-instance state like a class. Abstract classes suit shared fields and partial implementations; interfaces suit multiple inheritance of type.

STEP-BY-STEP

1. Abstract class: IS-A with shared code and state
2. Interface: CAN-DO contract; default/static/private methods since Java 8
3. Prefer composition over deep hierarchies

INTERVIEW KEYWORDS

Abstract class | Interface | Prefer composition over deep hierarchies

One-Line Interview Answer: An abstract class can share state and implementation; an interface defines a contract and supports multiple implementations.

Q3. Explain `equals()` and `hashCode()` contract and why it matters.

SIMPLE ANSWER

If two objects are equal via `equals()`, they must have the same `hashCode()`. `HashMap` and `HashSet` rely on this—if you override `equals()` without `hashCode()`, equal objects may land in different buckets, causing duplicates or failed lookups. In a product cache keyed by SKU, incorrect `hashCode()` can cause duplicate entries and wrong inventory counts.

STEP-BY-STEP

1. Equal objects → same `hashCode`
2. Required for `HashMap/HashSet`
3. Use IDE generate or `Objects.hash()`

REAL PROJECT EXAMPLE

Two Product objects with same SKU must return true for `equals()` and same `hashCode()` so `HashMap.get(sku)` finds the right entry.

INTERVIEW KEYWORDS

Equal objects → same `hashCode` | Required for `HashMap/HashSet` | Use IDE generate or `Objects.hash()`

One-Line Interview Answer: If two objects are equal via `equals()`, they must have the same `hashCode()`.

Q4. What is the difference between `==` and `equals()`?**SIMPLE ANSWER**

`==` compares references for objects (same memory location) and compares values for primitives. `equals()` compares logical value equality when overridden. For objects such as `String` and `Integer`, use `equals()` when you want to compare values. Use `==` for primitives and to check whether two object references point to the same object. Autoboxed `Integer` comparison outside the cache range (-128 to 127) is a common pitfall—prefer `equals()` for wrapper objects in business logic.

STEP-BY-STEP

1. Primitives: use `==` for value comparison.
2. Objects: `==` checks same reference; `equals()` checks logical equality.
3. `String` and wrapper types: prefer `equals()` for value comparison.
4. Always override `equals()` and `hashCode()` together for custom classes.
5. Use `Objects.equals(a, b)` to handle null safely.

INTERVIEW KEYWORDS

`==` | `equals()` | reference equality | value equality | `hashCode` contract

One-Line Interview Answer: Use `equals()` to compare object values; use `==` for primitives or to check if two references point to the same object.

Q5. Explain immutability and why `String` is immutable.**SIMPLE ANSWER**

Immutable objects cannot change after creation. They are safe to share between threads because their state cannot change after construction. `String` is immutable so it can be interned in the string pool and used safely as `HashMap` keys. For e-commerce, teams use immutable `Money` or `Address` value objects passed between services without accidental modification during checkout.

STEP-BY-STEP

1. Thread safety
2. Safe caching and hashing

3. Use records (Java 16+) for DTOs

REAL PROJECT EXAMPLE

record Money(BigDecimal amount, String currency) {} — immutable value object for order totals.

INTERVIEW KEYWORDS

Thread safety | Safe caching and hashing | Use records (Java 16+) for DTOs

One-Line Interview Answer: Immutable objects are safe to share between threads because their state cannot change after construction.

Q6. Checked vs unchecked exceptions—when to use which?

SIMPLE ANSWER

Checked exceptions extend `Exception` (but not `RuntimeException`) and the compiler forces you to handle or declare them. Unchecked exceptions extend `RuntimeException` and the compiler does not force handling. Not every checked exception is recoverable, and not every unchecked exception is a programming bug—choose based on API design and whether the caller can reasonably recover. In Spring Boot REST APIs, teams often use custom unchecked exceptions handled by `@ControllerAdvice`.

STEP-BY-STEP

1. Checked: compiler enforces catch or throws in signature.
2. Unchecked: extends `RuntimeException`; no forced handling.
3. Use checked when caller should explicitly handle (e.g., I/O).
4. Use unchecked for validation and business rule failures in REST layers.
5. Map exceptions consistently via `@ControllerAdvice`.

INTERVIEW KEYWORDS

checked exception | unchecked exception | `RuntimeException` | recovery | `@ControllerAdvice`

One-Line Interview Answer: Checked exceptions force handling at compile time; unchecked do not—choose based on whether the caller can recover.

Q7. Explain JVM memory areas briefly.

SIMPLE ANSWER

JVM memory includes Heap (Young and Old generation for objects), Metaspace (class metadata), Stack (per-thread method frames), PC registers, and native method stack. In production OutOfMemory scenarios, a heap dump reveals what objects retain memory—the fix depends on the cause: remove memory leaks, limit collection sizes, reduce cache footprint, add TTL, stream large results, release references, and only then consider heap tuning.

STEP-BY-STEP

1. Heap: where most application objects live (Young → Old after GC).
2. Stack: per-thread frames for method calls and local variables.
3. Metaspace: class metadata (replaced PermGen in Java 8+).
4. On OOM: capture heap dump and identify dominator objects.
5. Fix root cause before only increasing `-Xmx`.

INTERVIEW KEYWORDS

Heap | Stack | Metaspace | Young Gen | Old Gen | heap dump

One-Line Interview Answer: Heap holds objects; stack holds per-thread frames; heap dumps show what retains memory so you can fix the root cause.

JVM Memory Areas

Heap	Objects (Young Gen: Eden, Survivor → Old Gen)
Metaspace	Class metadata and static class info
Stack	Per-thread method frames and local variables
PC Register	Current instruction pointer per thread
Native Stack	Native method calls (JNI)

Young Gen → Minor GC → objects promoted to Old Gen → Full GC (more expensive)

Q8. Difference between method overloading and method overriding.**SIMPLE ANSWER**

Overloading means two or more methods in the same class share a name but differ in parameter list (type, count, or order). The compiler picks the correct method at compile time — compile-time polymorphism. Overriding means a subclass provides a new implementation for a method already defined in its parent class with the same signature. The JVM selects the implementation at runtime based on the actual object type — runtime polymorphism.

STEP-BY-STEP

1. Overloading: same class, different parameters, compile time
2. Overriding: subclass replaces parent method, runtime
3. Overriding requires inheritance and compatible return type

REAL PROJECT EXAMPLE

PaymentService.charge(amount) vs charge(amount, currency) is overloading; RazorpayGateway.process() overriding PaymentGateway.process() is overriding.

INTERVIEW KEYWORDS

Overloading | Overriding | Overriding requires inheritance and

Interview Tip: Mention @Override prevents accidental overload when you meant to override.

One-Line Interview Answer: Overloading uses the same method name with different parameters; overriding provides a new implementation in a subclass with the same signature.

Q9. Difference between throw and throws in Java.**SIMPLE ANSWER**

throw is a statement used inside a method body to explicitly raise an exception when a business rule fails — for example, throw new IllegalArgumentException("Invalid order quantity"). throws is part of a method signature and declares that the method may propagate a checked exception to its caller. Unchecked exceptions (RuntimeException) do not require throws. In Spring Boot REST layers, services typically throw custom runtime exceptions that @ControllerAdvice converts to HTTP responses.

STEP-BY-STEP

1. throw: creates and raises the exception
2. throws: declares possible checked exceptions
3. Prefer custom unchecked exceptions in service layer

INTERVIEW KEYWORDS

throw | throws | Prefer custom unchecked exceptions

Interview Tip: Give one line each with a validation example.

One-Line Interview Answer: throw is a statement used inside a method body to explicitly raise an exception when a business rule fails — for example, throw new IllegalArgumentException("Invalid order quantity").

Q10. What is garbage collection and when does the JVM run it?

SIMPLE ANSWER

Garbage Collection (GC) reclaims heap memory for objects unreachable from GC roots (live threads, static fields, JNI references). GC is triggered by collector heuristics and allocation/memory pressure—not only when the heap is literally full. Minor GC commonly involves the young generation; Full GC can involve the entire heap and is usually more expensive. Monitor GC logs and fix memory leaks before only increasing -Xmx.

STEP-BY-STEP

1. GC reclaims objects unreachable from GC roots.
2. Triggered by collector heuristics and allocation pressure.
3. Minor GC commonly targets the young generation.
4. Full GC can involve the entire heap and is usually more expensive.
5. Monitor GC logs before only increasing heap size.

INTERVIEW KEYWORDS

Triggered by heuristics and | Minor GC | Tune only after measuring

Interview Tip: Connect to production: mention analyzing GC logs after p99 latency spikes during high traffic.

One-Line Interview Answer: GC reclaims unreachable objects; Minor GC targets young generation, Full GC is more expensive and may pause the entire heap.

Part 2. Collections

Q11. How does HashMap work internally in Java?

SIMPLE ANSWER

HashMap stores key-value pairs in an array of buckets. The bucket index is derived from key.hashCode() after internal hash spreading (not simply hashCode % capacity). Collisions within a bucket use linked list or red-black tree (Java 8+ when threshold met). Default load factor 0.75 triggers resize. Average get/put is O(1); worst case O(log n) with treeification. Keys must implement equals/hashCode correctly.

STEP-BY-STEP

1. Hash spreading → bucket index
2. Collision: list/tree
3. Load factor 0.75
4. Not thread-safe

REAL PROJECT EXAMPLE

productMap.put(productId, product) — O(1) lookup during cart add-item flow.

INTERVIEW KEYWORDS

Hash spreading → bucket index | Collision | Load factor 0.75 | Not thread-safe

One-Line Interview Answer: HashMap stores key-value pairs in an array of buckets. The bucket index is derived from key.

Q12. HashMap vs Hashtable vs ConcurrentHashMap?

SIMPLE ANSWER

HashMap is not synchronized—fast for single-threaded use or external synchronization. Hashtable is legacy synchronized on every operation—avoid in new code. ConcurrentHashMap allows safe concurrent access and uses fine-grained synchronization and CAS techniques internally, giving better concurrency than Hashtable. Use ConcurrentHashMap when multiple threads read and write shared maps.

STEP-BY-STEP

1. HashMap: not thread-safe; fastest for single-threaded access.
2. Hashtable: legacy; synchronized on entire map—avoid.
3. ConcurrentHashMap: thread-safe with better concurrency.
4. Prefer ConcurrentHashMap for shared in-memory staging data.
5. Keep shared maps bounded or use external store for large data.

INTERVIEW KEYWORDS

HashMap | Hashtable | ConcurrentHashMap | thread-safe | CAS

One-Line Interview Answer: Use ConcurrentHashMap for thread-safe concurrent access—it scales better than Hashtable.

Q13. When would you use ArrayList vs LinkedList?

SIMPLE ANSWER

ArrayList is backed by a dynamic array— $O(1)$ random access, $O(n)$ insert/delete in the middle. LinkedList is a doubly linked list— $O(1)$ insert/delete at ends, $O(n)$ random access. In most backend applications, ArrayList is the practical default unless there is a specific reason to use LinkedList.

STEP-BY-STEP

1. ArrayList: fast get by index—good for paginated lists.
2. LinkedList: fast add/remove at head/tail only.
3. ArrayList uses less memory per element in typical cases.
4. Prefer ArrayList for typical service-layer collection use.
5. Choose LinkedList only with a measured performance reason.

INTERVIEW KEYWORDS

ArrayList | LinkedList | random access | insertion cost | backend default

One-Line Interview Answer: ArrayList is the practical default in backend code unless you have a specific reason for LinkedList.

Q14. Explain fail-fast vs fail-safe iterators.

SIMPLE ANSWER

Fail-fast iterators (ArrayList, HashMap) throw ConcurrentModificationException if the collection is structurally modified during iteration (except via iterator.remove()). CopyOnWriteArrayList iterates over a snapshot taken at iterator creation—safe for read-heavy config lists but may show stale data. ConcurrentHashMap iterators are weakly consistent—they reflect the table at some point in time and may miss concurrent updates without throwing.

STEP-BY-STEP

1. Fail-fast: CME on structural change
2. CopyOnWriteArrayList: snapshot iterator
3. ConcurrentHashMap: weakly consistent iterator

INTERVIEW KEYWORDS

Fail-fast | CopyOnWriteArrayList | ConcurrentHashMap

One-Line Interview Answer: Fail-fast iterators throw `ConcurrentModificationException` when structurally modified during iteration.

Q15. What is the difference between Set implementations: HashSet, LinkedHashSet, TreeSet?

SIMPLE ANSWER

HashSet: unordered, $O(1)$ add/contains—deduplicate cart line items by productId. LinkedHashSet: insertion order preserved—useful for ordered unique IDs. TreeSet: sorted (Red-Black tree), $O(\log n)$ —maintain sorted category IDs. Choose based on whether you need order or sorting, not just uniqueness.

STEP-BY-STEP

1. HashSet: no order
2. LinkedHashSet: insertion order
3. TreeSet: sorted

INTERVIEW KEYWORDS

HashSet | LinkedHashSet | TreeSet

One-Line Interview Answer: HashSet: unordered, $O(1)$ add/contains—deduplicate cart line items by productId.

Q16. How do Comparable and Comparator differ for sorting orders?

SIMPLE ANSWER

Comparable (compareTo) is implemented by the class itself—Order implements Comparable by orderDate. Comparator is external—sort orders by totalAmount without modifying Order class. In REST APIs we often use `Comparator.comparing(Order::getCreatedAt).reversed()` for latest-first listing without changing entity code.

STEP-BY-STEP

1. Comparable: natural ordering in class
2. Comparator: external, flexible
3. Java 8 method references simplify comparators

REAL PROJECT EXAMPLE

`orders.sort(Comparator.comparing(Order::getTotal).reversed())` — top orders first for admin dashboard.

INTERVIEW KEYWORDS

Comparable | Comparator | Java 8 method references

One-Line Interview Answer: Comparable defines the class natural ordering; Comparator provides external or custom sorting logic.

Part 3. Multithreading & Concurrency

Q17. Difference between process and thread.

SIMPLE ANSWER

A process has its own memory space; a thread is a lightweight unit within a process sharing heap memory. A Spring Boot app runs as one JVM process with multiple threads—Tomcat request threads, @Async worker threads, and Kafka consumer threads. Thread safety matters because they share OrderService singleton beans and cached data.

STEP-BY-STEP

1. Process: isolated memory
2. Thread: shared heap within process
3. Spring beans are singleton by default—watch shared mutable state

INTERVIEW KEYWORDS

Process | Thread | Spring beans are singleton by default

One-Line Interview Answer: A process has its own memory space; a thread is a lightweight unit within a process sharing heap memory.

Q18. Explain synchronized keyword and its limitations.**SIMPLE ANSWER**

synchronized ensures only one thread executes a block or method on the same monitor lock at a time. It may be used sparingly on legacy counters before moving to DB optimistic locking. Limitations: no timeout, no try-lock by default, and threads contending for the same monitor lock are blocked—ConcurrentHashMap and java.util.concurrent tools are preferred for scalability.

STEP-BY-STEP

1. Mutual exclusion
2. Can cause contention
3. Prefer java.util.concurrent for new code

INTERVIEW KEYWORDS

Mutual exclusion | Can cause contention | Prefer java.util.concurrent for new code

One-Line Interview Answer: synchronized provides mutual exclusion via a monitor lock, but contending threads block and scalability can suffer.

Q19. What is a thread pool and why use ExecutorService?**SIMPLE ANSWER**

Creating threads is expensive. ExecutorService manages a reusable thread pool. Spring @Async is not automatic—it delegates to a configured TaskExecutor (ThreadPoolTaskExecutor by default unless customized). Tomcat also uses thread pools for HTTP requests. Tune corePoolSize and maxPoolSize based on workload: CPU-bound vs I/O-bound tasks.

STEP-BY-STEP

1. Reuse threads via pools
2. @Async uses configured executor
3. Tomcat request threads are also pooled

REAL PROJECT EXAMPLE

executor.submit(() -> emailService.sendOrderConfirmation(order)) — offloads work from request thread.

INTERVIEW KEYWORDS

Reuse threads via pools | @Async uses configured executor | Tomcat request threads are also pooled

One-Line Interview Answer: Creating threads is expensive. ExecutorService manages a reusable thread pool.

Q20. Explain volatile keyword.**SIMPLE ANSWER**

volatile ensures visibility—when one thread writes a volatile field, other threads see the latest value immediately. It does not guarantee atomicity for compound actions like `count++`. Teams often use volatile for feature flags (e.g., maintenance mode) read by many request threads, but `AtomicInteger` for counters.

STEP-BY-STEP

1. Visibility guarantee
2. Not atomic for `i++`
3. Use `Atomic*` for counters

INTERVIEW KEYWORDS

Visibility guarantee | Not atomic for `i++` | Use `Atomic*` for counters

One-Line Interview Answer: volatile ensures visibility—when one thread writes a volatile field, other threads see the latest value immediately.

Q21. What are deadlocks and how do you prevent them?

SIMPLE ANSWER

Deadlock occurs when threads wait for each other's locks in a cycle. In a payment + inventory flow, Thread A locks inventory then payment while Thread B does the reverse—both stall. Prevention: consistent lock ordering, `tryLock` with timeout, minimize lock scope, avoid holding locks during external calls. Short DB transactions reduce lock duration but do not prevent deadlocks by themselves—design lock order and indexes carefully.

STEP-BY-STEP

1. Circular wait between threads
2. Consistent lock ordering
3. `tryLock` with timeout
4. `@Transactional` alone does not prevent deadlocks

INTERVIEW KEYWORDS

Circular wait between threads | Consistent lock ordering | `tryLock` with timeout | `@Transactional` alone does not

Interview Tip: Draw two threads and two locks to explain—interviewers appreciate clarity.

One-Line Interview Answer: Deadlock occurs when threads wait for each other's locks in a cycle.

Q22. Difference between `wait()`, `sleep()`, and `yield()`.

SIMPLE ANSWER

`wait()` releases the monitor lock and waits until the object is notified, interrupted, or otherwise awakened—used in producer-consumer patterns (rare in modern code). `sleep()` pauses the current thread for a duration without releasing locks—used for backoff retry. `yield()` hints the scheduler to run other threads—rarely used. In Spring Boot, prefer `LockSupport.parkNanos` or `ScheduledExecutorService` over raw `wait/sleep` in production retry logic.

STEP-BY-STEP

1. `wait`: releases lock, needs notify
2. `sleep`: holds locks
3. Prefer concurrent utilities over raw `wait/sleep`

INTERVIEW KEYWORDS

`wait` | `sleep` | Prefer concurrent utilities over

One-Line Interview Answer: `wait()` releases the monitor and waits for notification; `sleep()` pauses without releasing locks; `yield()` hints the scheduler.

Q23. How does `CompletableFuture` help in microservices?

SIMPLE ANSWER

`CompletableFuture` enables asynchronous composition and parallel execution. Product, Inventory, and Pricing services can be called in parallel during checkout with `CompletableFuture.allOf(...)`. Important nuance: if `supplyAsync` wraps blocking HTTP or JDBC, a thread is still blocked during that I/O—the benefit is composition and parallelism, not eliminating blocking unless using truly non-blocking clients. Always handle `exceptionally()` for timeouts and circuit breaker failures.

STEP-BY-STEP

1. Async composition and parallel calls
2. Blocking I/O inside `supplyAsync` still blocks a thread
3. Handle errors with `exceptionally()`

REAL PROJECT EXAMPLE

```
CompletableFuture.supplyAsync(() -> productClient.get(id)).thenCombine(priceFuture, this::buildLineItem)
```

INTERVIEW KEYWORDS

Async composition and parallel calls | Blocking I/O inside `supplyAsync` | Handle errors with `exceptionally()`

One-Line Interview Answer: `CompletableFuture` enables asynchronous composition and parallel execution.

Part 4. Java 8 / 11 / 17

Q24. Explain Lambda expressions and Functional Interfaces.

SIMPLE ANSWER

Lambda provides concise syntax for functional interfaces: `list.stream().filter(o -> o.isPaid())`. A functional interface has exactly one abstract method; `@FunctionalInterface` verifies and documents that constraint—it does not by itself make an interface functional. In order filtering: `orders.stream().filter(o -> o.getStatus() == OrderStatus.SHIPPED)`. If status is an enum, `==` is appropriate; if it is a String, use `equals()`.

STEP-BY-STEP

1. Lambdas implement single-method functional interfaces.
2. Use with Stream API: `filter`, `map`, `collect`.
3. Enum comparison: `==` is fine (e.g., `OrderStatus.SHIPPED`).
4. String comparison: use `equals()`, not `==`.
5. Method references (`Order::getTotal`) improve readability.

INTERVIEW KEYWORDS

Lambda | functional interface | Stream API | enum vs String | method reference

One-Line Interview Answer: Lambdas implement functional interfaces; `@FunctionalInterface` verifies the interface has one abstract method.

Q25. What are Stream API benefits and when not to use streams?

SIMPLE ANSWER

Stream API helps process collections in a clean, declarative way using operations such as filter, map, sorted, collect, and reduce. Streams are lazy—intermediate operations run only when a terminal operation (collect, reduce, forEach) is called. For example, to calculate total order amount, use map() and reduce() with BigDecimal. For primitive numbers, Java provides mapToInt(), mapToLong(), and mapToDouble()—there is no standard mapToBigDecimal(). Use streams when they improve readability; avoid them when logic becomes complicated, relies heavily on side effects, or involves blocking I/O.

STEP-BY-STEP

1. Create a stream from a collection (list.stream()).
2. Apply intermediate operations: filter, map, sorted.
3. Finish with a terminal operation: collect, reduce, or forEach.
4. For BigDecimal totals, map each item then reduce with BigDecimal::add.
5. Avoid DB calls or blocking I/O inside stream pipelines.

REAL PROJECT EXAMPLE

```
BigDecimal total = orders.stream().map(Order::getTotal).reduce(BigDecimal.ZERO, BigDecimal::add);
```

INTERVIEW KEYWORDS

Stream | filter | map | reduce | collect | lazy evaluation | readability

Interview Tip: If asked about BigDecimal, say map + reduce—not mapToBigDecimal, which does not exist in the standard API.

One-Line Interview Answer: Streams let you process collections declaratively with filter, map, and reduce—use them for readability, not for blocking I/O.

Q26. Optional—how do you use it properly in service layer?

SIMPLE ANSWER

Optional wraps a value that may be absent, forcing explicit handling instead of null checks scattered everywhere. In the service layer, repository methods often return Optional<Order> findById(id), and the service uses orElseThrow() -> new OrderNotFoundException(id) to map to HTTP 404. Optional is intended as a return type—not as method parameters or entity fields. Avoid Optional.get() in normal business code because it throws NoSuchElementException when empty; prefer orElse(), orElseGet(), or orElseThrow().

STEP-BY-STEP

1. Repository returns Optional when a record may not exist.
2. Service maps empty Optional to a domain exception or default.
3. Use orElseThrow for not-found cases in REST APIs.
4. Do not use Optional as fields or method parameters.
5. Avoid get() without checking—prefer safe alternatives.

INTERVIEW KEYWORDS

Optional | orElseThrow | null safety | return type | NoSuchElementException

One-Line Interview Answer: Optional makes absence explicit—use it as a return type and prefer orElseThrow over get().

Q27. What new features in Java 11 and 17 matter for backend devs?

SIMPLE ANSWER

Java 10 introduced local-variable type inference (var). Java 11 (LTS) adds HttpClient, String helpers (isBlank, lines), and runtime improvements. Java 17 (LTS) adds records, sealed classes, and pattern matching for instanceof. Production teams typically stay on LTS releases (11 or 17) for long-term support.

STEP-BY-STEP

1. Java 10: var (local type inference)
2. Java 11 LTS: HttpClient, String API
3. Java 17 LTS: records, sealed classes

REAL PROJECT EXAMPLE

`record OrderResponse(Long id, String status, BigDecimal total) {}` — replaces 20-line DTO boilerplate.

INTERVIEW KEYWORDS

Java 10 | Java 11 LTS | Java 17 LTS

One-Line Interview Answer: Java 11 added HttpClient and useful String APIs; Java 17 added records and sealed classes—both are important LTS releases for backend development.

Q28. Explain method references with an example.**SIMPLE ANSWER**

Method references are shorthand lambdas: `Order::getTotal` equivalent to `o -> o.getTotal()`. Types: static (`Integer::parseInt`), instance (`order::getStatus`), constructor (`ArrayList::new`). In Stream pipelines they improve readability: `orders.stream().map(Order::getCustomerId).distinct().collect(toList())`.

STEP-BY-STEP

1. Cleaner than lambda when direct call
2. `Order::getTotal` vs `o -> o.getTotal()`

INTERVIEW KEYWORDS

Cleaner than lambda when direct call | Order

One-Line Interview Answer: Method references are shorthand for lambdas when directly calling an existing method, such as `Order::getTotal`.

Q29. Difference between map() and flatMap() in streams.**SIMPLE ANSWER**

`map()` transforms each element one-to-one: `orders.stream().map(Order::getItems)` gives `Stream<List<Item>>`. `flatMap()` flattens nested structures: `orders.stream().flatMap(o -> o.getItems().stream())` gives `Stream<Item>`. Essential when aggregating all line items across orders for warehouse pick-list generation.

STEP-BY-STEP

1. map: 1-to-1
2. flatMap: flatten nested
3. Common in `Optional.flatMap` too

REAL PROJECT EXAMPLE

`List<Item> allItems = orders.stream().flatMap(o -> o.getItems().stream()).toList();`

INTERVIEW KEYWORDS

map | flatMap | Common in `Optional.flatMap` too

One-Line Interview Answer: `map()` transforms each element; `flatMap()` transforms and flattens nested collections into a single stream.

Q30. What are Java Records and Sealed Classes (Java 17)?

SIMPLE ANSWER

Records are compact, immutable data carriers: record OrderSummary(Long id, String status, BigDecimal total) auto-generates constructor, accessor methods (id(), status(), total())—not JavaBean getId(), equals, hashCode, and toString. Ideal for REST DTOs and Kafka payloads. Sealed classes restrict subtypes: sealed class PaymentResult permits Success, Failure. Use records for API responses and events, not JPA entities.

STEP-BY-STEP

1. Records: accessor methods like id(), not getId()
2. Sealed classes: controlled inheritance
3. Use records for DTOs/events, not JPA entities

INTERVIEW KEYWORDS

Records | Sealed classes | Use records for DTOs/events,

Interview Tip: Mention adopting records on Java 17 LTS for DTOs—shows practical adoption.

One-Line Interview Answer: Records are compact, immutable data carriers: record OrderSummary(Long id, String status, BigDecimal total) auto-generates constructor, accessor methods (id(), status(), total())—not JavaBean getId(), equals, hashCode, and toString.

Part 5. Spring & Spring Boot

Q31. What is Dependency Injection and how does Spring implement it?

SIMPLE ANSWER

DI means objects receive dependencies from outside rather than creating them—loose coupling and testability. Spring IoC container creates beans and injects via constructor (preferred), @Autowired field, or setter. OrderService typically receives OrderRepository and PaymentClient via constructor injection—easy to mock in @WebMvcTest without starting full context.

STEP-BY-STEP

1. Constructor injection preferred
2. IoC container manages lifecycle
3. Enables testing with mocks

INTERVIEW KEYWORDS

Constructor injection preferred | IoC container manages lifecycle | Enables testing with mocks

One-Line Interview Answer: DI means objects receive dependencies from outside rather than creating them—loose coupling and testability.

Q32. Explain Spring Bean lifecycle.

SIMPLE ANSWER

Spring bean lifecycle: Create → Dependency Injection → BeanPostProcessor callbacks → Initialization (@PostConstruct) → Ready for use → Destruction (@PreDestroy on shutdown). Singleton scope (default) creates one instance per container. Prototype scope creates a new instance each time the bean is requested—distinct from request scope, which is one instance per HTTP request.

STEP-BY-STEP

1. Container creates the bean instance.

2. Dependencies are injected (@Autowired / constructor).
3. BeanPostProcessor runs before/after initialization.
4. @PostConstruct or InitializingBean runs initialization logic.
5. Bean is ready; @PreDestroy runs on context shutdown.

INTERVIEW KEYWORDS

bean lifecycle | singleton | prototype | request scope | @PostConstruct

One-Line Interview Answer: Lifecycle is Create → Injection → Initialization → Ready → Destruction; prototype ≠ request scope.

Spring Bean Lifecycle

1. Instantiate bean
2. Populate properties (@Autowired)
3. BeanNameAware / BeanFactoryAware (if implemented)
4. @PostConstruct / afterPropertiesSet()
5. Bean ready for use
6. @PreDestroy / destroy() on context shutdown

Q33. @Component vs @Service vs @Repository vs @Controller?**SIMPLE ANSWER**

All are stereotype @Component variants. @Service marks business logic (OrderService). @Repository adds persistence exception translation (OrderRepository). @Controller/@RestController mark web layer. Functionally similar for component scanning, but semantic layering helps team navigation and AOP (e.g., @Transactional often on @Service).

STEP-BY-STEP

1. Same scanning, different semantics
2. @Repository: DB exception translation
3. @RestController = @Controller + @ResponseBody

INTERVIEW KEYWORDS

Same scanning, different semantics | @Repository | @RestController = @Controller +

One-Line Interview Answer: @Component is the base stereotype; @Service marks business logic, @Repository marks persistence, and @Controller handles web requests.

Q34. How does Spring Boot auto-configuration work?**SIMPLE ANSWER**

Spring Boot scans classpath, evaluates @ConditionalOnClass/@ConditionalOnMissingBean, and auto-configures beans (DataSource, Jackson, Tomcat). application.yml overrides defaults. For example, a config server module may exclude DataSourceAutoConfiguration when it has no database. Understanding conditions helps debug unexpected bean creation in multi-module builds.

STEP-BY-STEP

1. spring.factories / AutoConfiguration.imports
2. Conditional annotations
3. Override via custom @Configuration

INTERVIEW KEYWORDS

```
spring.factories / AutoConfiguration.imports | Conditional annotations | Override via custom
@Configuration
```

One-Line Interview Answer: Spring Boot scans classpath, evaluates `@ConditionalOnClass/@ConditionalOnMissingBean`, and auto-configures beans (DataSource, Jackson, Tomcat).

Q35. What are Spring Boot profiles and how did you use them?

SIMPLE ANSWER

Profiles (dev, staging, prod) load environment-specific config: application-prod.yml with real DB URLs, dev with H2. Activate via `SPRING_PROFILES_ACTIVE` in Docker/ECS. Feature flags and log levels differ per profile. Never commit prod secrets—use AWS Secrets Manager or env vars injected at deploy time.

STEP-BY-STEP

1. application-{profile}.yml
2. `SPRING_PROFILES_ACTIVE`
3. Secrets via env, not git

REAL PROJECT EXAMPLE

dev: H2 in-memory; prod: RDS PostgreSQL URL from environment variable.

INTERVIEW KEYWORDS

```
application-{profile}.yml | SPRING_PROFILES_ACTIVE | Secrets via env, not git
```

One-Line Interview Answer: Spring Profiles load environment-specific configuration such as dev, staging, and prod settings.

Q36. Explain `@Transactional`—propagation and rollback rules.

SIMPLE ANSWER

`@Transactional` defines a DB transaction boundary. Default propagation `REQUIRED` joins existing or creates new. Common bug: `@Transactional` on private method or self-invocation (`this.save()`) bypasses proxy—transaction does not start. Keep transactions short: external payment gateway calls should generally run outside long DB transactions (commit order first, use outbox/Saga, or async payment) to avoid holding locks during slow network I/O.

STEP-BY-STEP

1. `REQUIRED` default
2. Self-invocation bypasses proxy
3. Avoid long transactions around external payment calls
4. `rollbackFor` for checked exceptions when needed

REAL PROJECT EXAMPLE

`OrderService.createOrder() @Transactional` — saves order and items atomically; failure rolls back both.

INTERVIEW KEYWORDS

```
REQUIRED default | Self-invocation bypasses proxy | Avoid long transactions around | rollbackFor for
checked exceptions
```

One-Line Interview Answer: `@Transactional` defines a DB transaction boundary. Default propagation `REQUIRED` joins existing or creates new.

Q37. How do you handle exceptions globally in Spring Boot?

SIMPLE ANSWER

@ControllerAdvice with @ExceptionHandler returns consistent ErrorResponse JSON across all controllers. Map OrderNotFoundException → 404, ValidationException → 400, PaymentFailedException → 422 (Unprocessable Entity). HTTP 402 Payment Required has limited practical use—422 is the safer default for business/payment rule failures. Include traceId from MDC for log correlation. Never expose stack traces to clients in production.

STEP-BY-STEP

1. Create @ControllerAdvice class with @ExceptionHandler methods.
2. Map domain exceptions to appropriate HTTP status codes.
3. Use 422 for business/payment rule failures.
4. Include traceId and field-level errors in response body.
5. Log full exception server-side only.

INTERVIEW KEYWORDS

@ControllerAdvice | @ExceptionHandler | 422 | traceId | ErrorResponse

One-Line Interview Answer: Use @ControllerAdvice for consistent errors—422 for business failures, 404 for not found, 400 for validation.

Q38. What are @Primary and @Qualifier when multiple beans exist?**SIMPLE ANSWER**

When two or more beans implement the same interface (e.g., EmailSender and SmsSender both as NotificationSender), Spring needs help choosing. @Primary on one bean makes it the default when injection by type alone. @Qualifier("beanName") on injection point selects a specific named bean. Constructor injection with @Qualifier is preferred over field injection.

STEP-BY-STEP

1. @Primary: default bean for type
2. @Qualifier: explicit bean name
3. Prefer constructor injection

INTERVIEW KEYWORDS

@Primary | @Qualifier | Prefer constructor injection

Interview Tip: Explain the problem first (two implementations), then the two solutions.

One-Line Interview Answer: @Primary selects the default bean; @Qualifier selects a specific bean when multiple implementations exist.

Part 6. JPA & Hibernate**Q39. JPA vs Hibernate vs Spring Data JPA?****SIMPLE ANSWER**

JPA is the specification (Entity, EntityManager). Hibernate is the most popular JPA implementation. Spring Data JPA adds repository abstraction—OrderRepository extends JpaRepository with zero boilerplate CRUD and query methods. A typical stack uses Spring Data JPA on Hibernate against PostgreSQL—spec for portability, Spring Data for productivity.

STEP-BY-STEP

1. JPA: spec
2. Hibernate: implementation

3. Spring Data JPA: repository layer

INTERVIEW KEYWORDS

JPA | Hibernate | Spring Data JPA

One-Line Interview Answer: JPA is the specification, Hibernate is an implementation, and Spring Data JPA provides repository abstractions.

Q40. Explain Entity relationships: @OneToMany, @ManyToOne, @ManyToMany.

SIMPLE ANSWER

JPA models relationships between tables using annotations. Order @OneToMany List<OrderItem> items and OrderItem @ManyToOne Order order form a bidirectional link with mappedBy on the parent. @ManyToMany suits simple join tables (Order ↔ Promotion). In real applications, Many-to-Many is often modeled using a separate entity such as OrderPromotion because it gives better control over the relationship. Use cascade only when the child lifecycle is controlled by the parent—for example, Order may cascade persist/remove to OrderItem. Do not blindly use CascadeType.ALL.

STEP-BY-STEP

1. Place the foreign key on the @ManyToOne (many) side.
2. Use mappedBy on the inverse @OneToMany side.
3. Choose fetch type consciously—Lazy is default for collections.
4. Apply cascade only when parent owns child lifecycle.
5. Prefer a join entity instead of @ManyToMany when extra columns are needed.

REAL PROJECT EXAMPLE

Order 1—* OrderItem *—1 Product — standard e-commerce mapping.

INTERVIEW KEYWORDS

@OneToMany | @ManyToOne | mappedBy | cascade | fetch type | join entity

Interview Tip: Expect follow-up: "Why did you use CascadeType.ALL?"—answer with lifecycle ownership, not convenience.

One-Line Interview Answer: Model relationships with clear ownership of the FK, explicit fetch type, and cascade only when the parent controls the child lifecycle.

Q41. What is the N+1 problem and how do you fix it?

SIMPLE ANSWER

N+1 means one query loads N parent records, then N additional queries load each child—101 queries for 100 orders. Fix with JOIN FETCH in JPQL, @EntityGraph on repository methods, batch fetching (@BatchSize), or DTO projections for read APIs. JOIN FETCH is powerful but needs care with collection relationships and pagination—it can produce duplicate rows or inefficient queries, so test the generated SQL.

STEP-BY-STEP

1. Detect N+1 with SQL logging or Hibernate statistics.
2. Use JOIN FETCH or @EntityGraph for needed associations.
3. Consider @BatchSize for collection batch loading.
4. For list APIs, prefer DTO projections over loading full entity graphs.
5. Watch JOIN FETCH + pagination on collections—verify query plan.

REAL PROJECT EXAMPLE

```
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.customerId = :id")
```

INTERVIEW KEYWORDS

N+1 | JOIN FETCH | EntityGraph | DTO projection | @BatchSize

Interview Tip: N+1 is one of the most asked JPA questions—always mention JOIN FETCH and EntityGraph.

One-Line Interview Answer: N+1 is extra queries per row—fix with JOIN FETCH or EntityGraph, but validate SQL when paginating collections.

Q42. Difference between persist, merge, and save in Spring Data.**SIMPLE ANSWER**

persist (EntityManager) attaches new entity to persistence context. merge copies detached entity state into managed copy. Spring Data save() does persist-or-merge based on ID null or not. Bug pattern: load entity, close session, modify, save without merge—changes lost. Use @Transactional on service so entity stays managed throughout request.

STEP-BY-STEP

1. New entity: persist/save
2. Detached: merge needed
3. Keep work inside @Transactional

INTERVIEW KEYWORDS

New entity | Detached | Keep work inside @Transactional

One-Line Interview Answer: persist attaches a new entity, merge copies detached state into a managed entity, and save() in Spring Data chooses persist or merge.

Q43. Lazy vs Eager loading—best practices.**SIMPLE ANSWER**

Lazy loading means related data is fetched only when accessed. Eager loading fetches related data immediately with the parent. For backend applications, LAZY is generally preferred; explicitly fetch required data using JOIN FETCH, @EntityGraph, or DTO projections. Avoid EAGER fetching for collections by default—it can cause unnecessary queries, high memory use, and cartesian product issues.

STEP-BY-STEP

1. Default to LAZY for @OneToMany collections.
2. Fetch explicitly per use case (JOIN FETCH / EntityGraph).
3. Avoid global FetchType.EAGER on collections.
4. Use DTOs for read APIs that need limited fields.
5. Monitor generated SQL to confirm fetch behavior.

INTERVIEW KEYWORDS

Lazy | Eager | JOIN FETCH | EntityGraph | DTO projection | cartesian product

One-Line Interview Answer: Prefer LAZY by default and fetch associations explicitly only where the use case requires them.

Q44. What are first-level and second-level cache in Hibernate?**SIMPLE ANSWER**

First-level cache is the persistence context per transaction/session—automatic. Second-level cache is optional, session-factory scoped, and requires an explicit Hibernate cache provider (EhCache, Infinispan, Hazelcast)—Redis is

not the default L2 cache unless specifically integrated. Suitable for read-mostly reference data (Product catalog); avoid caching frequently updated Order entities.

STEP-BY-STEP

1. L1: session scope, automatic
2. L2: requires configured cache provider
3. Cache read-mostly reference data only

INTERVIEW KEYWORDS

L1 | L2 | Cache read-mostly reference data only

One-Line Interview Answer: L1 cache belongs to the persistence context/session; L2 cache is optional and shared across sessions.

Q45. Optimistic vs Pessimistic locking in JPA.

SIMPLE ANSWER

Optimistic locking (@Version) checks a version column at update time and fails if another transaction modified the row. Pessimistic locking (PESSIMISTIC_WRITE) locks the database row during read. Optimistic locking works well when conflicts are relatively infrequent. Pessimistic locking can help when conflicts are frequent and preventing concurrent modification matters more than throughput. The choice depends on workload and transaction design.

STEP-BY-STEP

1. Optimistic: add @Version field; retry on OptimisticLockException.
2. Pessimistic: lock row during read for critical updates.
3. Optimistic suits low-contention inventory or order updates.
4. Pessimistic suits high-contention financial adjustments.
5. Measure conflict rate before choosing.

REAL PROJECT EXAMPLE

@Version private Long version; — UPDATE fails if another thread updated stock first.

INTERVIEW KEYWORDS

optimistic locking | pessimistic locking | @Version | contention | retry

One-Line Interview Answer: Use optimistic locking when conflicts are rare; pessimistic when preventing concurrent modification is critical.

Part 7. REST API & Spring Security

Q46. REST principles you follow when designing APIs.

SIMPLE ANSWER

Resources as nouns (/orders, /products), HTTP verbs for actions (GET list, POST create, PATCH update). Keep APIs stateless where practical; JWT is one possible authentication mechanism. Proper status codes: 201 + Location header on create, 404 for missing order, 409 for duplicate payment. Version via /api/v1. Pagination with page, size, sort query params for order history.

STEP-BY-STEP

1. Nouns not verbs in URLs
2. Correct HTTP status codes
3. Pagination and filtering standards

REAL PROJECT EXAMPLE

GET /api/v1/orders?page=0&size=20&sort=createdAt,desc

INTERVIEW KEYWORDS

Nouns not verbs in URLs | Correct HTTP status codes | Pagination and filtering standards

One-Line Interview Answer: Design REST APIs with noun-based resources, correct HTTP verbs, stateless requests, and consistent status codes.

Q47. How do you validate request payloads in Spring Boot?**SIMPLE ANSWER**

Use @Valid on @RequestBody DTOs with Bean Validation annotations such as @NotNull, @Min, and @Email. Add custom validators for business rules like valid SKU format. Either handle BindingResult manually or let MethodArgumentNotValidException reach @ControllerAdvice for consistent field-level 400 responses. Keep validation on DTOs separate from JPA entities so API contracts stay independent of persistence.

STEP-BY-STEP

1. Define validation rules on the request DTO (@NotNull, @Size, @Email).
2. Add @Valid on @RequestBody in the controller method.
3. Use custom validators for business-specific rules.
4. Return field-level errors via @ControllerAdvice.
5. Keep API DTO separate from JPA entity.

INTERVIEW KEYWORDS

@Valid | Bean Validation | @ControllerAdvice | DTO | field errors

One-Line Interview Answer: Validate request DTOs with @Valid and Bean Validation; return consistent field errors from @ControllerAdvice.

Q48. Explain JWT authentication flow in Spring Security.**SIMPLE ANSWER**

User logs in with credentials → auth server validates → returns signed JWT (header.payload.signature). Client sends Authorization: Bearer <token> on each request. JwtAuthenticationFilter validates signature and expiry, loads authorities, sets SecurityContext. Stateless—no server session store. Use short-lived access tokens; store refresh tokens securely (HttpOnly cookie or secure mobile storage), implement rotation and revocation for logout and token compromise.

STEP-BY-STEP

1. Stateless auth via signed JWT
2. Short-lived access tokens
3. Refresh token rotation and secure storage

INTERVIEW KEYWORDS

Stateless auth via signed JWT | Short-lived access tokens | Refresh token rotation and

One-Line Interview Answer: The user logs in, receives a signed JWT, sends it with each request, and Spring Security validates the token before allowing access.

JWT Authentication Flow

```
[Client] --(1) login credentials--> [Auth Service]
[Auth Service] --(2) JWT access token--> [Client]
```

```
[Client] --(3) API request + Bearer JWT--> [Resource API]
[Resource API] --(4) verify signature/expiry--> [OK / 401]
```

Q49. Difference between authentication and authorization.

SIMPLE ANSWER

Authentication verifies who you are (login, JWT subject claim). Authorization verifies what you can do (@PreAuthorize("hasRole('ADMIN')") on DELETE /products). In marketplace, customer role can POST /orders; admin role can PATCH /orders/{id}/status. Spring Security filter chain handles authn; method security handles authz.

STEP-BY-STEP

1. Authn: identity
2. Authz: permissions
3. @PreAuthorize for fine-grained control

INTERVIEW KEYWORDS

Authn | Authz | @PreAuthorize for fine-grained control

One-Line Interview Answer: Authentication verifies who you are; authorization verifies what you are allowed to do.

Q50. How do you secure REST APIs beyond JWT?

SIMPLE ANSWER

Beyond JWT, secure APIs with HTTPS, rate limiting, CORS whitelisting, input validation, and parameterized queries to prevent SQL injection. CSRF protection depends on how auth is delivered: for stateless APIs where the token is sent in the Authorization header, CSRF is generally not needed the same way as cookie-based sessions. If authentication relies on cookies, CSRF protection becomes important. Audit logs and security headers add defense in depth.

STEP-BY-STEP

1. Enforce HTTPS and validate all inputs.
2. Apply rate limiting at gateway or application layer.
3. Whitelist CORS origins in production.
4. Match CSRF strategy to auth mechanism (header JWT vs cookies).
5. Log admin actions and apply least-privilege roles.

INTERVIEW KEYWORDS

HTTPS | rate limiting | CORS | CSRF | input validation | least privilege

One-Line Interview Answer: Layer HTTPS, validation, rate limits, and CORS—and match CSRF policy to whether auth uses cookies or Authorization headers.

Q51. What HTTP status codes do you return for common scenarios?

SIMPLE ANSWER

200 OK for successful GET, 201 Created for POST with body/link, 204 No Content for DELETE. 400 validation failure, 401 missing/invalid token, 403 valid token but insufficient role, 404 resource not found, 409 conflict (duplicate order idempotency key), 422 business rule failure, 500 unexpected server error with generic message to client.

STEP-BY-STEP

1. Be precise, not always 200
2. 409 for idempotency conflicts
3. Never leak internals on 500

INTERVIEW KEYWORDS

Be precise, not always 200 | 409 for idempotency conflicts | Never leak internals on 500

One-Line Interview Answer: Use 200/201/204 for success, 400 for validation, 401/403 for security, 404 for missing resources, 409 for conflicts, 422 for business failures, and 500 for unexpected errors.

Q52. Explain idempotency in payment and order APIs.

SIMPLE ANSWER

Idempotency is a behavioral property: repeating the same operation yields the same result. HTTP PUT and DELETE are idempotent by design. POST is not inherently idempotent, but APIs can implement idempotent create using an Idempotency-Key header. The server stores the idempotency key with the operation result; duplicate retries with the same key return the stored response without double charge. The same idempotency key should not be reused for a different request payload.

STEP-BY-STEP

1. Client sends Idempotency-Key header on POST create/payment.
2. Server stores key + request fingerprint + response.
3. Duplicate request with same key returns stored result.
4. Do not reuse the same key for a different operation.
5. Set expiry window (e.g., 24 hours) for stored keys.

REAL PROJECT EXAMPLE

Client sends Idempotency-Key: uuid on POST /payments — retry safe within 24h window.

INTERVIEW KEYWORDS

Idempotency-Key | stored response | POST idempotency | duplicate retry | payment safety

One-Line Interview Answer: Store idempotency key + result so retries are safe—never reuse the same key for a different request.

Q53. What is CORS and how do you resolve CORS errors from the backend?

SIMPLE ANSWER

Cross-Origin Resource Sharing (CORS) is a browser security mechanism. When a frontend at <https://shop.example.com> calls an API at <https://api.example.com>, the browser sends a preflight OPTIONS request. The API must respond with Access-Control-Allow-Origin or the browser blocks the response. Fix on Spring Boot: @CrossOrigin or global WebMvcConfigurer.addCorsMappings. In production, whitelist specific frontend domains.

STEP-BY-STEP

1. Browser-only restriction
2. Preflight OPTIONS for non-simple requests
3. Whitelist origins in production

INTERVIEW KEYWORDS

Browser-only restriction | Preflight OPTIONS for non-simple | Whitelist origins in production

Interview Tip: Check Network tab preflight and response headers first.

One-Line Interview Answer: CORS controls browser cross-origin requests; configure allowed origins and handle preflight OPTIONS requests correctly.

Part 8. Microservices & Design Patterns

Q54. Monolith vs Microservices—when to choose which?

SIMPLE ANSWER

Monolith suits small teams, simple domain, fast initial delivery. Microservices suit independent scaling (Product catalog vs Payment), team autonomy, and polyglot needs—but add network latency, distributed tracing, and deployment complexity. Teams split when release coupling and scaling bottlenecks hurt flash-sale performance—not because microservices are trendy.

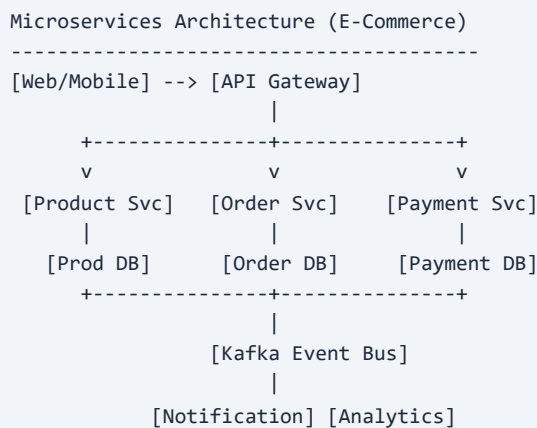
STEP-BY-STEP

1. Start monolith if unsure
2. Split on scaling/team boundaries
3. Operational cost is real

INTERVIEW KEYWORDS

Start monolith if unsure | Split on scaling/team boundaries | Operational cost is real

One-Line Interview Answer: Choose a monolith for simplicity and fast delivery; choose microservices when independent scaling, team ownership, or domain boundaries justify the added complexity.



Q55. What is an API Gateway and why use it?

SIMPLE ANSWER

An API Gateway is the single public entry point routing requests to backend services. It handles cross-cutting concerns: routing, SSL termination, rate limiting, and optional token validation. The gateway can perform initial authentication or JWT validation, but individual services should still enforce their own authorization and trust boundaries where required. Avoid putting business logic in the gateway.

STEP-BY-STEP

1. Clients call one public gateway URL.
2. Gateway routes /orders to Order service, /products to Product service.
3. Gateway may validate JWT format/expiry at the edge.
4. Each service still enforces role-based authorization.
5. Keep domain logic in services, not the gateway.

INTERVIEW KEYWORDS

API Gateway | routing | rate limiting | JWT validation | authorization boundary

One-Line Interview Answer: The gateway centralizes routing and cross-cutting concerns, but services must still enforce their own authorization.

Q56. How do microservices communicate?

SIMPLE ANSWER

Microservices communicate synchronously (REST, gRPC) or asynchronously (Kafka, RabbitMQ). Synchronous communication means the caller waits for a response—it does not automatically guarantee strong or immediate consistency. Asynchronous communication means the caller does not wait for downstream processing to finish; consistency is handled separately, often eventually. Choose sync when an immediate response is required; choose async for decoupling and resilience.

STEP-BY-STEP

1. Sync (REST): caller waits—use for queries needing immediate data.
2. Async (Kafka): fire event—use for notifications, analytics, indexing.
3. Consistency is a separate design decision from sync vs async.
4. Avoid long synchronous chains across many services.
5. Combine both patterns in the same system when appropriate.

INTERVIEW KEYWORDS

REST | Kafka | sync vs async | coupling | eventual consistency

One-Line Interview Answer: Sync means the caller waits; async means decoupled processing—consistency is designed separately from the communication style.

Q57. What is service discovery and how does it work?

SIMPLE ANSWER

Service discovery lets services register their network location and clients resolve services by name instead of hardcoded IPs. Examples include Eureka (Spring Cloud), Kubernetes DNS, or AWS Cloud Map. Depending on the platform and load-balancing mechanism, health checks can prevent unhealthy instances from receiving traffic—but behavior varies by implementation.

STEP-BY-STEP

1. Service registers its address on startup.
2. Client resolves service name to current instance list.
3. Load balancer or client-side routing picks a healthy instance.
4. Health checks mark failed instances as unavailable.
5. Platform determines exact routing and failover behavior.

INTERVIEW KEYWORDS

service discovery | Eureka | Kubernetes DNS | health check | load balancing

One-Line Interview Answer: Service discovery resolves services by name; health checks help keep traffic away from unhealthy instances, depending on the platform.

Q58. Explain Circuit Breaker pattern.

SIMPLE ANSWER

When downstream Payment service fails repeatedly, circuit opens—fail fast without waiting for timeout. After cooldown, half-open tries one request; success closes circuit. Resilience4j integrates with Spring Boot. During a

payment gateway outage, the order service can return graceful "payment temporarily unavailable" instead of hanging threads and cascading failure.

STEP-BY-STEP

1. Closed → Open → Half-Open
2. Resilience4j / Spring Cloud CircuitBreaker
3. Fallback responses

INTERVIEW KEYWORDS

Closed → Open → Half-Open | Resilience4j / Spring Cloud | Fallback responses

One-Line Interview Answer: Circuit Breaker moves from Closed to Open after repeated failures, then Half-Open for recovery testing before returning to Closed.

Circuit Breaker States

CLOSED (normal) -- failures exceed threshold --> OPEN (fail fast)

OPEN -- after wait duration --> HALF-OPEN (trial call)

HALF-OPEN -- success --> CLOSED | failure --> OPEN

Q59. What is the Saga pattern for distributed transactions?

SIMPLE ANSWER

Saga splits global transaction into local transactions with compensating actions. Choreography: services react to Kafka events. Orchestration: Saga orchestrator sends commands. Checkout Saga: create order → reserve inventory → charge payment; if payment fails, compensate by releasing inventory and marking order failed. No 2PC across microservice DBs.

STEP-BY-STEP

1. Compensating transactions
2. Choreography vs orchestration
3. Eventual consistency

INTERVIEW KEYWORDS

Compensating transactions | Choreography vs orchestration | Eventual consistency

One-Line Interview Answer: Saga splits global transaction into local transactions with compensating actions.

Saga (Orchestrated Checkout)

[Orchestrator] --> Create Order --> Reserve Stock --> Charge Payment

|
success

v
continue

|
success

v
continue

|
fail

v
Compensate: release stock, cancel order

Q60. How do you handle distributed tracing?

SIMPLE ANSWER

OpenTelemetry provides the standard for collecting traces, metrics, and logs. Micrometer Tracing integrates with Spring Boot and can export to backends such as Jaeger or Zipkin. TraceId and spanId propagate across HTTP headers and Kafka messages so one checkout request spans Gateway → Order → Product → Payment. Distributed tracing helps find slow downstream services that are hard to locate via log grep alone.

STEP-BY-STEP

1. Instrument services with OpenTelemetry / Micrometer Tracing.
2. Propagate traceId/spanId in HTTP and Kafka headers.
3. Export spans to Jaeger, Zipkin, or another backend.
4. Use traces to find cross-service latency bottlenecks.
5. Correlate traces with structured logs via traceId.

INTERVIEW KEYWORDS

OpenTelemetry | Micrometer Tracing | traceId | spanId | Jaeger | Zipkin

One-Line Interview Answer: OpenTelemetry + Micrometer Tracing propagates traceId across services to debug cross-service latency.

Q61. Explain Singleton pattern and how Spring uses it.**SIMPLE ANSWER**

Singleton ensures one instance per classloader. Spring beans are singleton scope by default—one OrderService shared by all request threads, so beans must be stateless or thread-safe. Prototype scope creates a new bean instance each time the container is asked for that bean—not per HTTP request. Request scope is separate: one instance per HTTP request. Avoid manual static getInstance() singletons in Spring—the container manages lifecycle.

STEP-BY-STEP

1. Default Spring scope: singleton (one instance per container).
2. Keep singleton beans stateless or thread-safe.
3. Prototype: new instance on each container request for that bean.
4. Request scope: one instance per HTTP request (different from prototype).
5. Let Spring manage scope—avoid manual singleton patterns.

INTERVIEW KEYWORDS

singleton | prototype | request scope | stateless bean | Spring scope

One-Line Interview Answer: Spring beans are singleton by default; prototype creates new instances on demand—distinct from request scope.

Q62. Factory vs Strategy pattern in payment integration.**SIMPLE ANSWER**

Factory creates objects without exposing creation logic—PaymentGatewayFactory returns Stripe or Razorpay gateway by config. Strategy defines interchangeable algorithms—PaymentStrategy interface with processPayment(). Spring @Qualifier or Map injection selects strategy at runtime based on customer region or merchant settings.

STEP-BY-STEP

1. Factory: object creation
2. Strategy: interchangeable behavior
3. Spring DI implements both naturally

INTERVIEW KEYWORDS

Factory | Strategy | Spring DI implements both naturally

One-Line Interview Answer: Factory creates objects without exposing creation logic—PaymentGatewayFactory returns Stripe or Razorpay gateway by config.

Q63. What is the Builder pattern and where do you use it?

SIMPLE ANSWER

Builder constructs complex objects step-by-step—Lombok @Builder on OrderDTO with 15 optional fields. Avoids telescoping constructors. Also used in test data setup: Order.builder().customerId(1L).items(list).build(). Records + builder less common; builder shines for mutable test fixtures and API response assembly.

STEP-BY-STEP

1. Readable object construction
2. Lombok @Builder
3. Great for tests

INTERVIEW KEYWORDS

Readable object construction | Lombok @Builder | Great for tests

One-Line Interview Answer: Builder constructs complex objects step-by-step—Lombok @Builder on OrderDTO with 15 optional fields.

Q64. CAP theorem in practical terms for e-commerce.

SIMPLE ANSWER

CAP applies during network partition: a distributed system cannot simultaneously guarantee full consistency and full availability. Product catalog may tolerate eventual consistency with checkout reservation for correction. Payment-critical operations often favor stronger consistency and explicit failure handling—but the exact design depends on business requirements, not a blanket "CP for all payments" label.

STEP-BY-STEP

1. CAP tradeoff applies during partition
2. Different services, different choices
3. Eventual consistency is common for async paths

INTERVIEW KEYWORDS

CAP tradeoff applies during partition | Different services, different choices | Eventual consistency is common

One-Line Interview Answer: CAP applies during network partition: a distributed system cannot simultaneously guarantee full consistency and full availability.

Q65. Eventual consistency—how do you explain it to product owners?

SIMPLE ANSWER

Eventual consistency means different parts of the system may temporarily show different data, but they converge over time. After an order is placed, the analytics dashboard may update in a few seconds while the order database shows the order immediately. Identify operations where users or business rules cannot tolerate conflicting state, and design those with appropriate transactional and consistency guarantees. Analytics, search indexes, and recommendations can often tolerate eventual consistency.

STEP-BY-STEP

1. Identify operations that cannot tolerate conflicting or stale state.
2. Design those paths with appropriate transactions and consistency guarantees.
3. Use async events for search, analytics, and recommendations.
4. Set SLAs on consumer lag and monitor offset lag.
5. Explain acceptable delay to stakeholders with concrete examples.

INTERVIEW KEYWORDS

eventual consistency | transactional guarantees | consumer lag | SLA | async paths

One-Line Interview Answer: Design strict consistency where business rules require it; let analytics and search catch up asynchronously.

Q66. Idempotent consumer pattern in event-driven systems.**SIMPLE ANSWER**

Kafka delivers at-least-once—consumers may see duplicate OrderPlaced events. Store processed eventId in DB with unique constraint; skip if already processed, or use upsert semantics. Kafka exactly-once semantics mainly cover Kafka-to-Kafka processing (transactions + idempotent producer)—external side effects (DB updates, email, payment) still need idempotency keys or transactional outbox patterns.

STEP-BY-STEP

1. At-least-once delivery requires dedup
2. Dedup by event ID or business key
3. Kafka EOS ≠ external side-effect exactly-once

INTERVIEW KEYWORDS

At-least-once delivery requires dedup | Dedup by event ID or business key | Kafka EOS ≠ external

One-Line Interview Answer: Kafka delivers at-least-once—consumers may see duplicate OrderPlaced events.

Part 9. Kafka & Messaging**Q67. What is Apache Kafka and why use it in microservices?****SIMPLE ANSWER**

Kafka is distributed commit log for high-throughput event streaming. Producers write to topics; consumers in groups read with offset tracking. Kafka is commonly used for OrderPlaced, PaymentCompleted events—decouples Order service from Email, Analytics, and Search indexers. Handles flash-sale spike better than synchronous fan-out REST calls.

STEP-BY-STEP

1. Durable log
2. Consumer groups for scale
3. Decoupling services

INTERVIEW KEYWORDS

Durable log | Consumer groups for scale | Decoupling services

One-Line Interview Answer: Kafka is distributed commit log for high-throughput event streaming.

Kafka Architecture

```
[Order Service] --produce--> [Topic: order-events] --consume--> [Notification Svc]
                                     |
                                     +--consume--> [Analytics Svc]
                                     +--consume--> [Search Indexer]
[ZooKeeper/KRaft] -- cluster metadata & controller
```

Q68. Explain Kafka topics, partitions, and consumer groups.**SIMPLE ANSWER**

Topic is a logical stream (order-events). Partitions parallelize throughput—partition by orderId so events for the same order go to the same partition and maintain ordering for that order. Consumer group: each partition is consumed by one consumer in the group; add consumers to scale up to partition count. 12 partitions, 4 consumers → each handles 3 partitions. More consumers than partitions sit idle.

STEP-BY-STEP

1. Partition = parallelism + ordering unit
2. Consumer group shares load
3. Key-based partitioning for order events

INTERVIEW KEYWORDS

Partition = parallelism + ordering unit | Consumer group shares load | Key-based partitioning for order events

One-Line Interview Answer: Partition by business key (orderId) to preserve event ordering per order while scaling consumers in groups.

Q69. At-least-once vs at-most-once vs exactly-once semantics.**SIMPLE ANSWER**

At-most-once: commit offset before processing—may lose messages on crash. At-least-once: process then commit—may duplicate if crash occurs between processing and commit. Kafka producer idempotence prevents duplicate Kafka records caused by certain producer retries—it does not make an entire business operation exactly-once. Kafka transactions cover Kafka-to-Kafka flows; external DB, payment, and email side effects still need application-level idempotency.

STEP-BY-STEP

1. At-most-once: fast but may lose messages.
2. At-least-once: common default—handle duplicates in consumers.
3. Idempotent producer: dedupes Kafka retries, not business logic.
4. Kafka transactions: Kafka-to-Kafka exactly-once scope.
5. External side effects: use idempotency keys or outbox pattern.

INTERVIEW KEYWORDS

at-least-once | idempotent producer | Kafka transactions | business idempotency | offset commit

One-Line Interview Answer: Producer idempotence stops duplicate Kafka records from retries; business exactly-once still needs application-level dedup.

Q70. What is consumer lag and how do you monitor it?**SIMPLE ANSWER**

Lag = difference between latest offset and consumer committed offset—backlog indicator. High lag means consumers cannot keep up—scale consumers, optimize processing, or increase partitions. Alert when consumer lag exceeds thresholds during sales. Burrow or Kafka UI plus Prometheus metrics for dashboards.

STEP-BY-STEP

1. Monitor lag per consumer group
2. Scale out or optimize handler
3. Dead letter topic for poison messages

INTERVIEW KEYWORDS

Monitor lag per consumer group | Scale out or optimize handler | Dead letter topic for poison messages

One-Line Interview Answer: Lag = difference between latest offset and consumer committed offset—backlog indicator.

Q71. How do you handle poison messages in Kafka?**SIMPLE ANSWER**

A poison message fails processing repeatedly. If the consumer keeps retrying it synchronously without moving past it, progress for that partition can stall. After N retries with backoff, send the message to a Dead Letter Topic (DLT) with original payload and error metadata. Alert on DLT volume. Spring Kafka `@RetryableTopic` can automate retry delays and DLT routing. Replay from DLT manually after fixing the bug.

STEP-BY-STEP

1. Detect repeated failures on the same message.
2. Retry with exponential backoff (limited attempts).
3. Route to Dead Letter Topic after max retries.
4. Alert on DLT message count.
5. Fix bug and replay DLT messages if appropriate.

INTERVIEW KEYWORDS

poison message | Dead Letter Topic | retry backoff | `@RetryableTopic` | partition progress

One-Line Interview Answer: Retry poison messages with backoff, then move to a Dead Letter Topic so the partition can keep progressing.

Q72. Kafka vs RabbitMQ—when to choose Kafka?**SIMPLE ANSWER**

Kafka excels at high-throughput event streaming, durable log retention, replay from offset, and multiple independent consumer groups reading the same stream. RabbitMQ excels at traditional message queues, flexible routing (exchanges, bindings), and task distribution patterns. Choose based on business requirements—throughput, replay, routing complexity—not a fixed latency rule.

STEP-BY-STEP

1. Kafka: event log, replay, high volume, multiple consumers.
2. RabbitMQ: task queues, routing patterns, request-reply.
3. Evaluate retention, ordering, and consumer model needs.
4. Both can coexist in the same architecture.
5. Do not assume Kafka is always slower or RabbitMQ always faster.

INTERVIEW KEYWORDS

Kafka | RabbitMQ | event streaming | message queue | replay | routing

One-Line Interview Answer: Kafka for durable event streams and replay; RabbitMQ for flexible routing and task queues—choose by requirement.

Q73. How do you ensure message ordering in Kafka?**SIMPLE ANSWER**

Ordering guaranteed only within a partition. Use same message key (orderId) so all events for one order go to same partition. Do not increase partitions expecting global order. `PaymentCompleted` must follow `OrderCreated` for same

orderId—both keyed by orderId. Cross-order ordering is not guaranteed nor needed.

STEP-BY-STEP

1. Key determines partition
2. Order per key, not globally
3. Single partition = global order but no scale

INTERVIEW KEYWORDS

Key determines partition | Order per key, not globally | Single partition = global

One-Line Interview Answer: Ordering guaranteed only within a partition. Use same message key (orderId) so all events for one order go to same partition.

Part 10. Redis & Caching

Q74. What is Redis and common use cases in Java backend?

SIMPLE ANSWER

Redis is an in-memory data store used as cache, session store, rate limit counter, and pub/sub broker. Common uses: cache product details by SKU (with TTL), JWT blacklist on logout, API rate limiting with INCR + EXPIRE. Redis typically offers very low latency depending on network and deployment. Redis can implement distributed locking, but lock correctness requires TTL, unique ownership, and safe release.

STEP-BY-STEP

1. Cache: reduce DB load for read-heavy data (products, config).
2. Session store: fast session lookup across instances.
3. Rate limiting: INCR + EXPIRE counters per client/IP.
4. Pub/sub: lightweight messaging (not a Kafka replacement).
5. Distributed locks: only with TTL, unique value, safe release.

INTERVIEW KEYWORDS

cache | session | rate limit | low latency | distributed lock | TTL

One-Line Interview Answer: Redis speeds up reads with cache and counters—use distributed locks only with TTL and safe ownership.

Q75. Explain cache-aside pattern.

SIMPLE ANSWER

App checks cache first; on miss, read DB, write to cache, return. On update, write DB then invalidate cache (not update cache directly—race conditions possible). If the DB transaction succeeds but cache invalidation fails, TTL or event-based repair can act as a safety mechanism. ProductService: getBySku checks Redis, on miss loads from repository and populates cache.

STEP-BY-STEP

1. Read: cache → DB → populate
2. Write: DB then invalidate
3. Watch stampede on expiry

INTERVIEW KEYWORDS

Read | Write | Watch stampede on expiry

One-Line Interview Answer: Cache-aside: read cache first, on miss load DB; on write update DB then invalidate cache, with TTL as safety net.

Cache-Aside Pattern

```
-----
READ: App --> Cache? --hit--> return
      |
      miss
      v
      DB --> write Cache --> return

WRITE: App --> DB --> invalidate Cache
```

Q76. How do you integrate Redis with Spring Boot?

SIMPLE ANSWER

Spring Boot commonly uses spring-boot-starter-data-redis with the Lettuce client. @Cacheable("products") on getProduct(sku) with RedisCacheManager and TTL in config covers typical caching. RedisTemplate handles custom operations like counters and locks. Lettuce uses a non-blocking connection model—pooling is not automatically required and should not be added blindly; configure pooling only for specific blocking or connection-isolation needs.

STEP-BY-STEP

1. Add spring-boot-starter-data-redis dependency.
2. Configure Redis host, port, and TTL per cache name.
3. Use @Cacheable for declarative caching.
4. Use RedisTemplate for locks, counters, custom ops.
5. Add Actuator health indicator; tune pooling only if needed.

REAL PROJECT EXAMPLE

```
@Cacheable(value = "products", key = "#sku") public Product getBySku(String sku) { ... }
```

INTERVIEW KEYWORDS

Lettuce | Spring Cache | @Cacheable | RedisTemplate | TTL | connection model

One-Line Interview Answer: Spring Boot + Lettuce covers most Redis needs; add connection pooling only when workload justifies it.

Q77. Cache invalidation strategies when product price updates.

SIMPLE ANSWER

TTL alone can serve stale prices during promotions. On admin price update: delete Redis key product:{sku} after DB commit. For bulk updates, publish ProductUpdated events for cross-node invalidation. Write-then-invalidate is common, but race conditions can still cause brief staleness—use short TTL as a safety net. Versioned keys (product:v2:{sku}) can reduce thundering herd on mass invalidation.

STEP-BY-STEP

1. Invalidate on write
2. Race conditions possible—TTL as safety net
3. Event-driven invalidation across nodes

INTERVIEW KEYWORDS

Invalidate on write | Race conditions possible | Event-driven invalidation across nodes

One-Line Interview Answer: TTL alone can serve stale prices during promotions. On admin price update: delete Redis key `product:{sku}` after DB commit.

Q78. What is cache stampede and how to prevent it?

SIMPLE ANSWER

Cache stampede (thundering herd) happens when many requests hit an expired cache key at once—all miss and hammer the database. Prevention includes mutex lock during rebuild (Redis SET key NX with TTL), TTL jitter, or pre-warming before expiry. For production distributed locks: use a unique lock value, set TTL, release only if you still own the lock, and avoid deleting another process's lock.

STEP-BY-STEP

1. Many concurrent cache misses hit the database simultaneously.
2. Use SET NX EX to acquire lock with expiration.
3. Only one thread rebuilds cache; others wait or retry.
4. Store unique lock owner ID; release with compare-and-delete.
5. Add TTL jitter to spread expiry times.

INTERVIEW KEYWORDS

cache stampede | SET NX | TTL | distributed lock | lock ownership | jitter

One-Line Interview Answer: Prevent stampedes with short-lived locks that have unique owner values, TTL, and safe release.

Q79. Redis persistence: RDB vs AOF—do you need it for cache?

SIMPLE ANSWER

RDB snapshots the dataset periodically; AOF records write operations—actual durability depends on AOF fsync policy (always, everysec, no). Whether persistence is required depends on whether losing Redis data on restart is acceptable. Pure cache can tolerate cold start (TTL repopulates from DB). Session or counter data may need persistence if loss is not acceptable. Set maxmemory and eviction policy for cache workloads.

STEP-BY-STEP

1. Ask: is losing Redis data on restart acceptable?
2. Cache-only: persistence often optional; TTL repopulates.
3. Sessions/counters: consider AOF if data loss is costly.
4. Choose fsync policy based on durability vs performance needs.
5. Configure maxmemory and eviction (e.g., allkeys-lru).

INTERVIEW KEYWORDS

RDB | AOF | fsync policy | data loss tolerance | eviction policy

One-Line Interview Answer: Choose Redis persistence based on whether losing data on restart is acceptable for that use case.

Part 11. Database & SQL

Q80. Clustered vs Non-clustered index.

SIMPLE ANSWER

SQL Server and MySQL InnoDB support clustered indexes where index order may determine physical row storage. PostgreSQL does NOT have SQL Server-style clustered indexes—a PRIMARY KEY creates a B-tree index, but rows

remain in heap storage. Secondary indexes are separate B-tree structures pointing to row locations. Index columns used in WHERE, JOIN, and ORDER BY on large tables—but over-indexing slows writes.

STEP-BY-STEP

1. PostgreSQL PK = B-tree index, not physical row order.
2. SQL Server/InnoDB clustered index behaves differently.
3. Secondary indexes point to row locations.
4. Index filter/join/sort columns on large tables.
5. Balance read speed vs write overhead.

INTERVIEW KEYWORDS

clustered index | PostgreSQL | B-tree | heap storage | secondary index

One-Line Interview Answer: PostgreSQL PK is a B-tree index on heap storage—not SQL Server-style clustered physical ordering.

```
Database Indexing (Concept)
-----
Table: orders (heap storage)
Index on customer_id (B-Tree): customer_id --> row pointer
Query: WHERE customer_id = 42 --> index seek (fast)
      vs full table scan (may be expensive on large tables)
```

Q81. Explain ACID properties with order placement example.

SIMPLE ANSWER

Atomicity: order + items save together or rollback. Consistency: FK constraints ensure valid product_id. Isolation controls how concurrent transactions see each other's changes and helps prevent unwanted concurrency anomalies—for example, concurrent checkouts should not see partial inventory updates. Durability: committed order survives DB crash (WAL). @Transactional defines the transaction boundary; the database engine provides ACID guarantees.

STEP-BY-STEP

1. Atomicity, Consistency, Isolation, Durability
2. @Transactional = boundary; DB provides guarantees

INTERVIEW KEYWORDS

Atomicity, Consistency, Isolation, Durability | @Transactional = boundary; DB

One-Line Interview Answer: ACID ensures transactions are atomic, consistent, isolated, and durable—the DB enforces guarantees within the @Transactional boundary.

Q82. What are isolation levels and which do you use?

SIMPLE ANSWER

Isolation levels control which concurrent changes a transaction can observe. READ UNCOMMITTED, READ COMMITTED (PostgreSQL default), REPEATABLE READ, and SERIALIZABLE offer different trade-offs. Higher isolation can reduce certain anomalies but may increase contention or affect concurrency—the effect varies by database engine. Choose the level based on the transaction's consistency requirements, not as a one-size-fits-all rule.

STEP-BY-STEP

1. Understand anomalies: dirty read, non-repeatable read, phantom read.
2. PostgreSQL default is READ COMMITTED.
3. Higher levels reduce anomalies but may increase locking.
4. Behavior differs between PostgreSQL, MySQL, and SQL Server.

5. Choose per transaction based on consistency needs.

INTERVIEW KEYWORDS

isolation level | READ COMMITTED | anomalies | contention | database-specific

One-Line Interview Answer: Isolation levels control what concurrent changes you see—choose based on consistency needs and database behavior.

Q83. Inner join vs left join—example with orders and customers.

SIMPLE ANSWER

INNER JOIN orders o JOIN customers c ON o.customer_id = c.id returns only orders with valid customer. LEFT JOIN customers returns all orders even if customer deleted (orphan orders for audit). Admin report of all orders including archived accounts uses LEFT JOIN; customer-facing order history uses INNER JOIN.

STEP-BY-STEP

1. INNER: matching rows only
2. LEFT: all left table rows
3. Choose based on orphan handling

INTERVIEW KEYWORDS

INNER | LEFT | Choose based on orphan handling

One-Line Interview Answer: INNER JOIN returns only matching rows; LEFT JOIN returns all rows from the left table plus matching rows from the right table.

Q84. How do you optimize a slow SQL query in production?

SIMPLE ANSWER

Use EXPLAIN or EXPLAIN ANALYZE to understand the query plan. A sequential scan is not automatically bad—for small tables or queries returning a large percentage of rows, it may be the correct plan. Check whether filters, joins, sorting, or excessive row counts cause the problem. Add or modify indexes only when the execution plan and workload justify them. Also consider pagination, archiving, and avoiding SELECT *.

STEP-BY-STEP

1. Run EXPLAIN ANALYZE on the slow query.
2. Identify expensive operations: seq scan, sort, nested loop.
3. Evaluate whether an index helps this specific access pattern.
4. Rewrite correlated subqueries as JOINS where appropriate.
5. Paginate large result sets and archive old data.

INTERVIEW KEYWORDS

EXPLAIN ANALYZE | query plan | sequential scan | index | pagination

One-Line Interview Answer: Start with EXPLAIN ANALYZE—a seq scan is not always wrong; index only when the plan and workload justify it.

Q85. Database normalization vs denormalization.

SIMPLE ANSWER

Normalization (3NF) reduces redundancy—separate products, categories, order_items. Denormalization duplicates data for read speed—order snapshot stores product_name at purchase time so price changes don't alter history. E-commerce orders denormalize line item price; catalog stays normalized.

STEP-BY-STEP

1. Normalize for writes/consistency
2. Denormalize for read performance
3. Snapshot order line data intentionally

INTERVIEW KEYWORDS

Normalize for writes/consistency | Denormalize for read performance | Snapshot order line data intentionally

One-Line Interview Answer: Normalize to reduce redundancy and improve consistency; denormalize selectively when read performance or historical snapshots require it.

Q86. SQL injection prevention in Java applications.**SIMPLE ANSWER**

Never concatenate user input into SQL strings. JPA/JPQL named parameters and Spring Data method parameters are safe. JDBC use PreparedStatement. Audit native queries carefully—@Query with :param only. OWASP ZAP scan in CI catches unsafe patterns. One legacy report used string concat—replaced with parameterized query in security sprint.

STEP-BY-STEP

1. Parameterized queries always
2. Audit native SQL
3. JPA default is safe if no string concat

INTERVIEW KEYWORDS

Parameterized queries always | Audit native SQL | JPA default is safe if no string concat

One-Line Interview Answer: Never concatenate user input into SQL strings. JPA/JPQL named parameters and Spring Data method parameters are safe.

Part 12. AWS, Docker & CI/CD**Q87. How do you containerize a Spring Boot app with Docker?****SIMPLE ANSWER**

Use a multi-stage Dockerfile: a build stage (Maven/Gradle) and a slim runtime stage containing only what the application needs to run. Modern Java container images use optimized JDK/runtime bases rather than a separate traditional JRE install. Copy the JAR, expose port 8080, add HEALTHCHECK on /actuator/health, and run as non-root. Push to a container registry; orchestrator pulls and runs with env vars and secrets.

STEP-BY-STEP

1. Build stage: compile and package with Maven/Gradle.
2. Runtime stage: slim Java runtime image only.
3. Copy JAR; EXPOSE 8080; HEALTHCHECK /actuator/health.
4. Run as non-root user for security.
5. Push to ECR/registry; deploy via ECS/Kubernetes.

REAL PROJECT EXAMPLE

```
docker build -t order-service:1.2.0 . && docker push 123456.dkr.ecr.ap-south-1.amazonaws.com/order-service:1.2.0
```

INTERVIEW KEYWORDS

multi-stage build | slim runtime image | HEALTHCHECK | non-root | container registry

One-Line Interview Answer: Multi-stage Docker builds keep compile tools out of the production image—use a slim Java runtime base.

Q88. Explain CI/CD pipeline for microservices.

SIMPLE ANSWER

Developer pushes to Git → Jenkins/GitHub Actions runs unit tests, integration tests (Testcontainers), SonarQube scan → build Docker image → push to ECR → deploy to staging → smoke tests → manual/auto promote to prod with rolling update. Failed health check triggers rollback to previous task definition.

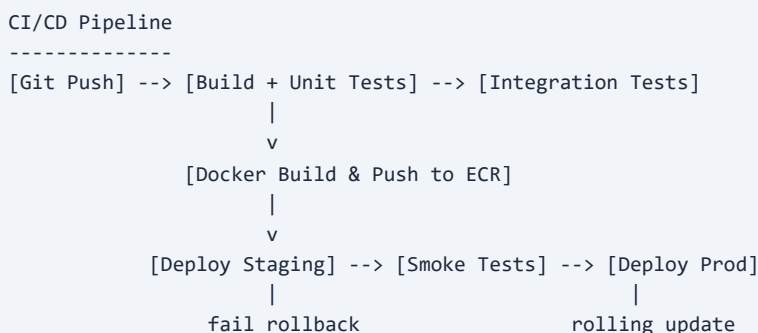
STEP-BY-STEP

1. Automated test gate
2. Immutable Docker artifacts
3. Rollback strategy

INTERVIEW KEYWORDS

Automated test gate | Immutable Docker artifacts | Rollback strategy

One-Line Interview Answer: Developer pushes to Git → Jenkins/GitHub Actions runs unit tests, integration tests (Testcontainers), SonarQube scan → build Docker image → push to ECR → deploy to staging → smoke tests → manual/auto promote to prod with rolling update.



Q89. Which AWS services are commonly used to deploy a Java backend?

SIMPLE ANSWER

Common AWS services for Java backend deployment: ECS Fargate (run containers without managing servers), RDS PostgreSQL (OLTP database), ElastiCache Redis (caching), Application Load Balancer (traffic routing), ECR (container registry), S3 (file storage), Secrets Manager (credentials), CloudWatch (logs and alarms), and IAM roles (service-to-service auth without hardcoded keys). MSK or self-managed Kafka on EC2 for event streaming where needed.

STEP-BY-STEP

1. ECS Fargate: run Docker containers at scale.
2. RDS: managed relational database (PostgreSQL).
3. ElastiCache: managed Redis for caching.
4. ALB: load balancing and health checks.
5. ECR + Secrets Manager + CloudWatch: deploy, secrets, monitoring.

INTERVIEW KEYWORDS

ECS Fargate | RDS | ElastiCache | ALB | ECR | Secrets Manager | CloudWatch

One-Line Interview Answer: Typical AWS stack: ECS Fargate, RDS, ElastiCache, ALB, ECR, Secrets Manager, and CloudWatch.

Q90. What is blue-green or rolling deployment?

SIMPLE ANSWER

Rolling: gradually replace old tasks with new version—default in many orchestrators, minimal extra infra. Blue-green: full parallel stack (blue live, green new), switch traffic via load balancer or deployment platform, fast rollback by switching back to blue depending on the platform and traffic-routing mechanism. Rolling suits most releases; blue-green helps major migrations needing quick revert.

STEP-BY-STEP

1. Rolling: incremental replacement
2. Blue-green: traffic switch via LB/platform
3. Health checks gate traffic shift

INTERVIEW KEYWORDS

Rolling | Blue-green | Health checks gate traffic shift

One-Line Interview Answer: Rolling replaces instances incrementally; blue-green switches traffic between full environments for fast rollback when the platform supports it.

Q91. How do you manage secrets in AWS for Spring Boot?

SIMPLE ANSWER

Store in AWS Secrets Manager or SSM Parameter Store. ECS/Kubernetes task definition maps secrets to env vars injected at runtime. Spring reads `SPRING_DATASOURCE_PASSWORD` from environment—never in application-`prod.yml` in git. Rotate credentials in Secrets Manager; whether restart is required depends on how the app loads secrets (env vars at startup vs dynamic refresh). Local dev uses `.env` excluded from git.

STEP-BY-STEP

1. Secrets Manager / SSM
2. Inject as env vars
3. Restart may or may not be required after rotation

INTERVIEW KEYWORDS

Secrets Manager / SSM | Inject as env vars | Restart may or may

One-Line Interview Answer: Store in AWS Secrets Manager or SSM Parameter Store. ECS/Kubernetes task definition maps secrets to env vars injected at runtime.

Q92. Docker Compose for local microservices development.

SIMPLE ANSWER

`docker-compose.yml` spins PostgreSQL, Redis, Kafka, and optionally dependent services locally. Spring profiles point to localhost containers. Developers run Order service in IDE for debugging while infra runs in Compose. Matches prod topology enough to catch integration issues before CI—Testcontainers used in automated tests similarly.

STEP-BY-STEP

1. Local parity with prod infra
2. Compose for dependencies
3. Testcontainers in CI

INTERVIEW KEYWORDS

Local parity with prod infra | Compose for dependencies | Testcontainers in CI

One-Line Interview Answer: `docker-compose.yml` spins PostgreSQL, Redis, Kafka, and optionally dependent services locally.

Part 13. Production & Troubleshooting

Q93. How do you debug a production issue like 502 errors?

SIMPLE ANSWER

Check load balancer target health and container/task status first. Review logs for OOM or startup failure. Trace a sample request through gateway to service. Correlate with recent deploys—rollback if needed. HTTP status distinction: 502 Bad Gateway (proxy received invalid response from upstream), 503 Service Unavailable (overloaded or maintenance), 504 Gateway Timeout (upstream did not respond in time). Possible 502 causes include unhealthy upstream instances, connection failures, invalid upstream responses, or proxy misconfiguration.

STEP-BY-STEP

1. 502: bad/invalid upstream response from proxy perspective.
2. 503: service unavailable or overloaded.
3. 504: upstream timeout.
4. Check health checks → logs → traces → recent deploys.
5. Rollback if customer impact and deploy correlation exists.

INTERVIEW KEYWORDS

502 | 503 | 504 | upstream health | proxy | rollback

One-Line Interview Answer: 502 means bad upstream response—check health, logs, traces, and recent deploys before assuming a single cause.

Q94. What metrics do you monitor for Spring Boot APIs?

SIMPLE ANSWER

Golden signals: latency (p95/p99), traffic (RPS), errors (5xx rate), saturation (CPU, thread pool, DB connections). Micrometer exports to Prometheus; Grafana dashboards alert on SLO breaches. Business metrics: orders per minute, payment failure rate. Alert on symptom (high 5xx) not only cause (CPU)—CPU can be fine while DB is slow.

STEP-BY-STEP

1. Latency, traffic, errors, saturation
2. Actuator + Prometheus
3. Business KPIs alongside tech metrics

INTERVIEW KEYWORDS

Latency, traffic, errors, saturation | Actuator + Prometheus | Business KPIs alongside tech metrics

One-Line Interview Answer: Golden signals: latency (p95/p99), traffic (RPS), errors (5xx rate), saturation (CPU, thread pool, DB connections).

Q95. How do you analyze OutOfMemoryError in JVM?

SIMPLE ANSWER

Enable `-XX:+HeapDumpOnOutOfMemoryError` and analyze the dump with Eclipse MAT or VisualVM. Check whether the issue is heap memory, metaspace, direct/native memory, or thread-related memory before assuming heap is the problem. Common heap causes: unbounded cache, loading entire large result sets into memory, or reference leaks. Fix the root cause before only increasing `-Xmx`.

STEP-BY-STEP

1. Identify OOM type: heap, metaspace, direct memory, threads.
2. Capture heap dump on OutOfMemoryError.

3. Analyze dominator tree in MAT for largest retained objects.
4. Fix leaks, bound caches, paginate large queries.
5. Increase heap only after fixing the root cause.

INTERVIEW KEYWORDS

OutOfMemoryError | heap dump | metaspace | MAT | memory leak | dominator tree

One-Line Interview Answer: Identify which memory area failed, analyze the heap dump, fix leaks—then tune heap if still needed.

Q96. Explain connection pool exhaustion and fix.**SIMPLE ANSWER**

HikariCP maintains a pool of database connections (e.g., max 20). Each request flows: Client → Service → Repository → HikariCP → PostgreSQL. If all connections are busy, new requests wait at HikariCP until timeout. Common causes: long @Transactional methods, slow queries, or connection leaks. Fix by shortening transactions, tuning pool size vs DB max_connections, and enabling leak detection.

STEP-BY-STEP

1. Request enters service and repository layer.
2. HikariCP lends a connection from the pool.
3. If all connections are in use, the request waits.
4. Wait exceeds timeout → "Connection pool exhausted" error.
5. Fix: shorter transactions, tune pool, enable leak detection.

REAL PROJECT EXAMPLE

Symptom: "Timeout: pool empty" in logs during peak — root cause 30s report inside @Transactional.

INTERVIEW KEYWORDS

HikariCP | connection pool | transaction scope | leak detection | timeout

One-Line Interview Answer: Pool exhaustion means all DB connections are held—shorten transactions and tune HikariCP before raising limits blindly.

Connection Pool Flow

Request → Service → Repository → HikariCP → PostgreSQL

All connections busy:

New Request → HikariCP → No connection → Wait → Timeout

Q97. How do you perform zero-downtime database migration?**SIMPLE ANSWER**

Use the expand-contract pattern so database changes never break the currently running application during a rolling deployment. Add a nullable column first, deploy backward-compatible application code, backfill data, then switch reads and writes, and only later remove the old column. Version migration scripts in Flyway/Liquibase and run them in CI/CD before app deploy.

STEP-BY-STEP

1. Old application still running on current schema.
2. Add new nullable column (expand)—old code ignores it.
3. Deploy compatible application that writes old + new columns.

4. Backfill existing rows with new column values.
5. Switch reads to new column, then drop old column later (contract).

INTERVIEW KEYWORDS

expand-contract | backward compatibility | Flyway | rolling deployment | migration

One-Line Interview Answer: Never make a DB change that breaks the running app—expand first, migrate data, contract later.

Zero-Downtime Migration (Expand-Contract)

 Old App → Add new column → Deploy compatible App
 → Backfill data → Write old + new → Switch reads
 → Remove old column later

Rule: never break the currently running app during rolling deploy.

Q98. Log aggregation and correlation in microservices.**SIMPLE ANSWER**

Structured JSON logs (Logback + logstash encoder) shipped to CloudWatch or ELK enable centralized search. MDC can carry correlation/trace IDs and carefully selected request identifiers—avoid putting sensitive user data in logs unless there is a legitimate need and appropriate masking/access controls. Propagate `traceld` from gateway through Feign/Kafka headers to correlate one `orderId` across all services.

STEP-BY-STEP

1. Use structured JSON logs for machine parsing.
2. Put `traceld/correlationId` in MDC for every request.
3. Propagate IDs across HTTP and Kafka calls.
4. Avoid logging sensitive PII without masking.
5. Centralize logs for cross-service search during incidents.

INTERVIEW KEYWORDS

structured logging | MDC | `traceId` | correlation | PII masking

One-Line Interview Answer: Use structured logs with `traceld` in MDC—propagate across services and avoid logging sensitive data.

Q99. How do you load test before a flash sale?**SIMPLE ANSWER**

Use Gatling or k6 to simulate realistic checkout flows at 2–3× expected peak traffic. Monitor DB CPU, connection pools, Kafka lag, and Redis memory during the test. Define SLOs based on business requirements—for example, target p99 latency and acceptable error rate (500 ms / 0.1% are examples, not universal rules). Fix bottlenecks found before the event.

STEP-BY-STEP

1. Script realistic user journeys (browse → cart → checkout).
2. Ramp load to 2–3× expected peak.
3. Monitor latency, errors, DB, cache, and messaging.
4. Define pass criteria from business SLOs.
5. Fix bottlenecks and re-test before go-live.

INTERVIEW KEYWORDS

load testing | k6 | Gatling | SLO | p99 latency | bottleneck

One-Line Interview Answer: Load-test at peak multiples with business-defined SLOs—not arbitrary universal latency targets.**Q100. Post-mortem process after a production incident.****SIMPLE ANSWER**

Run a blameless post-mortem soon after a significant incident, while details are still fresh. Structure: Detection → Mitigation → Root Cause → Corrective Action → Prevention → Follow-up. Document timeline, impact duration, what went well, and action items with owners. Example: payment double-charge from missing idempotency key—add validation and integration tests. Track actions to completion.

STEP-BY-STEP

1. Detection: how the issue was noticed (alerts, customers, monitoring).
2. Mitigation: what stopped customer impact (rollback, feature flag).
3. Root cause: technical and process reasons.
4. Corrective action: immediate fixes deployed.
5. Prevention: tests, alerts, design changes to avoid recurrence.
6. Follow-up: verify action items are completed.

INTERVIEW KEYWORDS

blameless post-mortem | root cause | corrective action | prevention | action items

Interview Tip: Show maturity by describing a real incident with structured learning—not hiding failures.**One-Line Interview Answer:** Post-mortems follow Detection → Mitigation → Root Cause → Corrective Action → Prevention → Follow-up.**Part A. Complete Java Backend Development → Deployment Flow****End-to-End Flow (Quick Reference)**

```

Requirement
  ↓
HLD / LLD (System Design)
  ↓
Database Design
  ↓
API Design
  ↓
Spring Boot Project Setup
  ↓
Client → API Gateway → Controller
  ↓
DTO + Validation → Service → Repository → Database
  ↓
(Entity ↔ Repository ↔ Database – persistence layer)
  ↓
Security (JWT) → Redis → Kafka
  ↓
Unit Testing → Integration Testing
  ↓
Git / Pull Request → Code Review
  ↓
CI Pipeline → Build JAR
  ↓
Docker Image → Container Registry (ECR)
  ↓

```

```

Deploy Staging → Smoke Testing
↓
Production Deployment
↓
Load Balancer → Application
↓
Database / Redis / Kafka
↓
Monitoring: Logs + Metrics + Traces
↓
Alerting → Rollback if required

```

What Happens at Each Stage

1. Requirement & HLD/LLD: understand business flow and architecture before coding.
2. Database & API design: tables, indexes, REST contracts, status codes.
3. Project setup: Entity, DTO, Repository, Service, Controller layers.
4. Cross-cutting: Security (JWT), Redis cache, Kafka events where needed.
5. Testing: unit tests for logic; integration tests with Testcontainers.
6. Git/PR → CI: build, test, Docker image, push to registry.
7. Staging smoke test → production deploy with health checks.
8. Monitoring and alerting; rollback if metrics degrade.

Part B. Where Does the Code Go?

Layer-by-Layer Guide

```

Controller      → receives HTTP request, validates input
DTO             → request/response structure (API contract)
Service         → business logic, @Transactional, orchestration
Repository      → database access (Spring Data JPA)
Entity          → maps to database table (persistence model)
Mapper          → converts Entity ↔ DTO
Integration/Client → external APIs (payment gateway, email)
Kafka Producer  → publishes async events
Kafka Consumer  → processes events from topics
Redis           → caching / fast temporary data
Config/Security → application.yml, JWT, security beans
Exception Handler → @ControllerAdvice for consistent errors
Tests           → unit (mocked) + integration (Testcontainers)

```

Request Flow (Build Order)

```

Client
↓
API Gateway
↓
Controller
↓
DTO + Validation
↓
Service
↓
Repository
↓
Database

Service also calls:
├─→ Redis (cache)
├─→ Kafka Producer (events)
└─→ External API (payment, email)

```

Persistence Layer (Separate Concern)

Entity ↔ Repository ↔ Database
(maps table) (JPA access) (PostgreSQL)

DTOs are used at the API boundary – not passed directly to Repository.

Cache Flow

Service → Redis → Cache Hit? → Return
↓ Miss
Database → Write Redis → Return

Async Event Flow

Order Service → Kafka Producer → Order Topic
↓
↓ ↓ ↓
Email Inventory Analytics
Consumer Consumer Consumer

Part C. Clean Architecture for Spring Boot

Layers and Rules

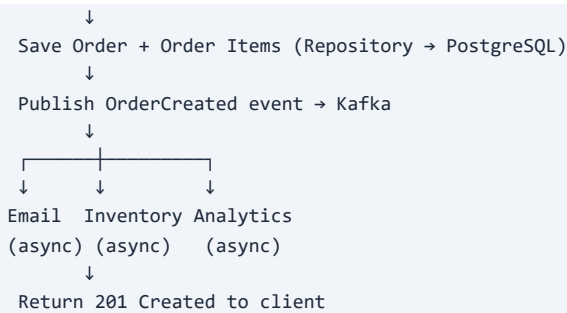
Rule of thumb: dependencies point inward—Controller depends on Service, Service depends on Repository, not the reverse.

1. Controller Layer: HTTP mapping, @Valid on DTOs, no business logic or direct DB access.
2. Service Layer: business rules, transactions, calls to repository/Kafka/Redis/external APIs.
3. Repository Layer: data access only—Spring Data JPA interfaces or custom queries.
4. Entity Layer: JPA-mapped domain objects tied to database tables—not returned directly in APIs.
5. DTO Layer: API contracts (request/response)—stable even when entities change.
6. Mapper Layer: converts Entity ↔ DTO—keeps layers decoupled.
7. Infrastructure Layer: Kafka producers/consumers, Redis clients, payment gateway clients.
8. Configuration Layer: beans, profiles, security, cache, and messaging setup.
9. Exception Handling: @ControllerAdvice maps exceptions to consistent HTTP responses.
10. Testing: unit tests mock repositories; integration tests use Testcontainers.

Part D. Complete Order Creation Flow

POST /api/v1/orders — Step by Step

```
POST /api/v1/orders
↓
API Gateway
↓
JWT Validation
↓
OrderController (validate CreateOrderRequest DTO)
↓
OrderService
↓
Validate request → Check Product → Check Inventory
↓
@Transactional
```



Who Does What

Sync steps: validation, inventory check, DB save, HTTP response. Async steps: email, analytics, search indexing via Kafka.

1. Controller: receives HTTP request, validates DTO, returns response.
2. Service: business logic, inventory check, transaction boundary.
3. Repository: INSERT order and order_items rows.
4. @Transactional: ensures order + items save atomically.
5. Kafka: async notification to email, inventory, analytics services.
6. Redis: cache product/inventory lookups where appropriate (not order writes).
7. Security: JWT proves identity; @PreAuthorize checks permissions.
8. Docker + CI/CD: packages and deploys the service; AWS runs containers.
9. Monitoring: traces and metrics detect slow steps or failures.

Part E. Deployment Strategies

1. Rolling Deployment

Gradually replace old instances with new ones. Good for normal releases with minimal extra infrastructure.

1. Orchestrator replaces one instance at a time.
2. Health checks gate traffic to new instances.
3. Advantage: no duplicate infrastructure cost.
4. Risk: old and new versions run simultaneously—must stay backward compatible.
5. Rollback: deploy previous image version.

```

Old v1:  v1  v1  v1
Deploy:  v1  v1  v2
Continue: v1  v2  v2
Done:    v2  v2  v2
  
```

2. Blue-Green Deployment

Run two full environments; switch traffic when green is verified. Good for fast rollback.

1. Deploy new version to green environment.
2. Run smoke tests against green.
3. Switch load balancer traffic to green.
4. Advantage: fast rollback by switching traffic back to blue, depending on platform.
5. Risk: requires double infrastructure during switch.

```

Load Balancer
|
  
```

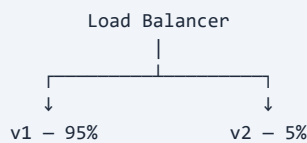


Test GREEN → Switch traffic → GREEN live
 Problem? → Switch back → BLUE live

3. Canary Deployment

Route a small percentage of traffic to the new version first; expand gradually if metrics are healthy.

1. Deploy v2 alongside v1.
2. Route 5% of traffic to v2.
3. Monitor error rate, latency, and business KPIs.
4. Gradually increase v2 traffic if healthy.
5. Rollback: redirect all traffic back to v1.



Monitor: error rate, latency, CPU, memory, business metrics
 Expand: 5% → 25% → 50% → 100%

Part F. CI/CD Pipeline

Standard Pipeline Flow

1. Git: every change goes through pull request and code review.
2. CI (Jenkins/GitHub Actions): build JAR, run tests automatically.
3. Docker: immutable artifact—same image from staging to prod.
4. Registry (ECR): stores versioned images.
5. Deploy: ECS/Kubernetes pulls image, injects secrets from Secrets Manager.
6. Load balancer: routes traffic only to healthy instances.
7. Monitoring: alerts on error rate and latency after deploy.

```

Git Push
↓
Build + Unit Tests
↓
Integration Tests (Testcontainers)
↓
Static Analysis (SonarQube)
↓
Security Scan
↓
Docker Build → Push to ECR
↓
Deploy Staging
↓
Smoke Tests
↓
Approval / Promotion
↓
Deploy Production
↓
Health Check → Monitoring
  
```



Rollback if required

Part G. Production Troubleshooting Flow

General Pattern

Symptom → Check metrics → Check logs → Check traces → Identify bottleneck → Mitigate → Fix root cause → Deploy → Monitor

Common Issues

1. 500 Internal Server Error: check application logs and stack traces; recent deploy correlation.
2. 502 Bad Gateway: upstream returned invalid response—check target health and container status.
3. 503 Service Unavailable: overloaded or maintenance—check capacity and circuit breakers.
4. 504 Gateway Timeout: upstream too slow—check DB, external APIs, and thread pools.
5. Slow API: traces show bottleneck; check N+1 queries, missing indexes, cache misses.
6. High CPU / Memory: profiler or heap dump; unbounded caches or large result sets.
7. OutOfMemoryError: heap dump analysis; fix leaks before increasing -Xmx.
8. Connection pool exhaustion: shorten transactions; check for leaks and slow queries.
9. Kafka consumer lag: scale consumers or optimize handler; check for poison messages.
10. Redis failure: fall back to DB; check memory, eviction policy, and connectivity.
11. Deployment failure: rollback to previous image; verify health checks and secrets.
12. High traffic: scale horizontally; verify cache, DB connection limits, and rate limiting.

Part H. Production Checklist

Before Go-Live

1. Development: layered architecture, validation, exception handling, idempotency for payments.
2. Testing: unit + integration tests pass; load test against business SLOs.
3. Database: migrations backward compatible; indexes on filter/join columns.
4. Security: HTTPS, JWT, role-based access, secrets in Secrets Manager—not git.
5. Deployment: Docker image with health check; CI/CD pipeline with rollback plan.
6. Monitoring: metrics (latency, errors, saturation), structured logs, distributed traces.
7. Alerting: alert on symptoms (5xx rate, p99 latency) not only CPU.
8. Runbook: document rollback steps, on-call escalation, and post-mortem process.